Natural Language Processing (NLP)

Vector Semantics & Embeddings

Embeddings

By:

Dr. Zohair Ahmed

www.youtube.com/@ZohairAI Subscribe

www.begindiscovery.com

Vector Semantics & Embeddings

Word Meaning

Let's begin with some linguistic background on word meaning

Embeddings

- Embeddings are numerical representations of words (or sentences) that capture their meanings in a way computers can understand.
- Words with similar meanings get similar numbers (vectors).

Why Early NLP Word Representations Were “Bad”

Before word embeddings (like Word2Vec), NLP used **very simple representations** of words:

- **Words were just strings**
  - "dog" is just the characters d-o-g
  - "cat" is just c-a-t
  - The computer doesn’t “know” anything else.
- **Or we used indices**
  - We created a vocabulary list like:
  - To the computer:

👉 **dog = 537**

👉 **cat = 891**

- That’s it. No meaning. No relationships. Just **IDs**, Like roll numbers in a class.
- This means the model **doesn’t know**:
  - dog and cat are both animals
  - dog is closer to wolf than to phone
  - life is different from LIFE
- All words look equally far apart.

Why Early NLP Word Representations Were “Bad”

⚠ The Problem

- This classical representation **has no meaning** built in.
- ❌ **No similarity**
  - The computer doesn’t know:
  - “dog” is similar to “puppy”
  - “king” is related to “queen”
  - All it sees are ID numbers.
- ❌ **No context**
  - The word “bank” (river bank) and “bank” (financial bank) look identical.
- ❌ **No relationships**
  - You can’t do reasoning like:
  - DOG → MAMMAL
  - There is no structure, no semantics, nothing.

So… What Do We Want Instead?

- We need representations that capture:

Similarity

- Words with similar meanings should be nearby (vectors close together)

Context

- Words should reflect how they are used in text ("apple" close to "fruit", "tree")

Relationships

- Embeddings allow:
  king − man + woman ≈ queen

What do words mean?

- N-gram or text classification methods we've seen so far
  - Words are just strings (or indices w_i in a vocabulary list)
  - That's not very satisfactory!
- Introductory logic classes:
  - The meaning of "dog" is DOG; cat is CAT

    $\forall x\ \mathrm{DOG}(x) \longrightarrow \mathrm{MAMMAL}(x)$
- Old linguistics joke by Barbara Partee in 1967:
  - Q: What's the meaning of life?
  - A: LIFE
- That seems hardly better!

Desiderata:
What Should a Theory of Word Meaning Do?

- Desiderata is a fancy Latin word that simply means: “Things we want

1. Capture Similarity
   - Words with similar meanings should be close (dog–cat, car–truck).
2. Capture Differences
   - Different senses or meanings should be distinguishable (bank💰 ≠ bank🏞️).
3. Handle Compositionality
   - Meanings should combine to form larger meanings
   - (e.g., red car, eat apples).
4. Reflect Context
   - A word’s meaning should depend on how it is used in sentences
   - (run a race vs run a company).
5. Support Inference / World Knowledge
   - Allow reasoning like:
   - dog → animal,
   - apple → fruit,
   - teacher → person.
6. Learn From Data
   - Work automatically from large text corpora without manual labeling.
7. Be Computable
   - Easy to use in algorithms, machine learning, and NLP tasks.
8. Be Generalizable
   - Work across tasks like translation, classification, QA, retrieval, etc.

Lemmas and senses

Sense / Concept

- A sense = one specific meaning of a word.
- A word can have many senses.

Example:

- mouse (animal) = Sense 1
- mouse (computer device) = Sense 2
- These are different senses of “mouse.”

Lemma

- A lemma = the dictionary form of a word.

Example:

- run → run, runs, running, ran (all belong to lemma RUN)

Polysemy

- A lemma is polysemous if it has multiple related or distinct senses.
- So:
  - mouse is polysemous
  - because it has two meanings.

Relations between senses: Synonymy

- Synonyms have the same meaning in some or all contexts.
  - filbert / hazelnut
  - couch / sofa
  - big / large
  - automobile / car
  - vomit / throw up
  - water / H_20

Even if two words have very similar meanings, they usually differ in:

- Politeness
  - die vs. pass away
- Slang / Informality
  - child vs. kid
- Register (formal vs informal)
  - purchase (formal) vs buy (informal)
- Genre / context
  - scientific vs everyday words
  - (cardiovascular vs heart-related)
- Because of these differences, no two words mean **EXACTLY the same** in all contexts.

The Linguistic Principle of Contrast

Difference in form → difference in meaning

Abbé Gabriel Girard 1718

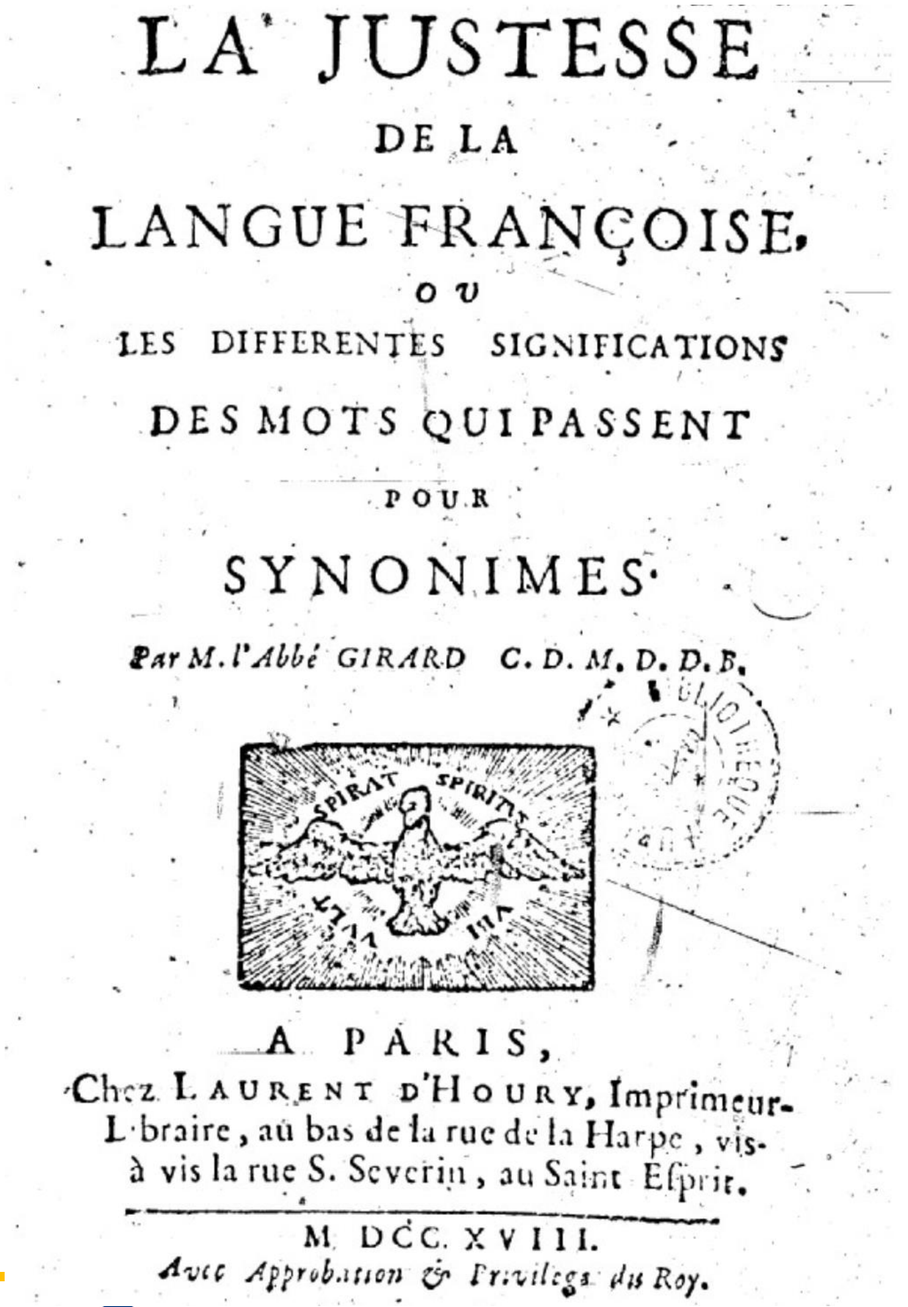
Re: "exact" synonyms

" je ne crois pas qu'il y ait de-
mot ſynonime dans aucune
Langue. Je le dis par con-"

[I do not believe that there is a synonymous word in any language]

- Even words that are very close (like big and large) are not identical:
- “big mistake” (common)
- “large mistake” (sounds wrong)
- So they differ in usage and meaning shades.

Thanks to Mark Aronoff!


LA JUSTESSE
DE LA
LANGUE FRANÇOISE,
OU
LES DIFFERENTES SIGNIFICATIONS
DES MOTS QUI PASSENT
POUR
SYNONIMES.
Par M. l'Abbé GIRARD C. D. M. D. D. B.
SPIRAT SPIRITV
A PARIS,
Chez LAURENT D'HOURY, Imprimeur-Libraire, au bas de la rue de la Harpe, vis-à-vis la rue S. Severin, au Saint Eſprit.
M. DCC. XVIII.
Avec Approbation & Privilege du Roy.

Relation: Similarity

- Words with similar meanings. Not synonyms, but sharing some element of meaning
  - car, bicycle
  - cow, horse
- Cat is not a synonym of dog, but cats and dogs are certainly similar words.
- The notion of word similarity is very useful in larger semantic tasks.
- Knowing how similar two words are can help in computing how similar the meaning of two phrases or sentences
- A very important component of natural language understanding tasks like question answering, paraphrasing, and summarization.

Ask humans how similar 2 words are

- One way of getting values for word similarity is to ask humans to judge how similar one word is to another.
- A number of datasets have resulted from such experiments.
- For example, the SimLex-999 dataset (Hill et al., 2015 gives values on a scale from 0 to 10, from near-synonyms (vanish, disappear) to pairs that scarcely seem to have anything in common (hole, agreement):
- Must Read its Part of Exams: https://aclanthology.org/J15-4004/

word1	word2	similarity
vanish	disappear	9.8
behave	obey	7.3
belief	impression	5.95
muscle	bone	3.65
modest	flexible	0.98
hole	agreement	0.3

SimLex-999 dataset (Hill et al., 2015)

Relation: Word relatedness

- Also called "word association"
- Words can be related in any way, perhaps via a semantic frame or field
  - coffee, tea: **similar**
  - coffee, cup: **related**, not similar
- The meaning of two words can be related in ways other than similarity.
- One such class of connections is called word relatedness (Budanitsky and Hirst, 2006), also traditionally called word association in psychology.
- Coffee is similar to tea. But coffee is not similar to cup; they share practically no features (coffee is a plant or a beverage, while a cup is a manufactured object with a particular shape).
- But coffee and cup are clearly related; they are associated by co-participating in an everyday event (the event of drinking coffee out of a cup).
- Similarly scalpel and surgeon are not similar but are related
  - eventively (a surgeon tends to make use of a scalpel).

Semantic field

- A semantic field is a set of words which cover a particular semantic domain and carry structured relations with each other.
- Words that
  - cover a particular semantic domain
  - carry structured relations with each other.
- **Hospitals**
  - *surgeon*, *scalpel*, *nurse*, *anaesthetic*, *hospital*
- **restaurants**
  - *waiter*, *menu*, *plate*, *food*, *menu, chef*
- **Houses**
  - *door*, *roof*, *kitchen*, *family*, *bed*

Relation: Antonymy

- Antonymy = when two senses (meanings of words) are opposites along one specific feature.
- Antonyms are opposites with respect to ONE feature, They differ in exactly one dimension of meaning.
- long ↔ short
  - They differ only in length (feature = LENGTH).
  - Everything else is the same:
  - both describe physical measurement
  - both apply to time or space
  - both are adjectives
  - Only the length feature flips.
- fast ↔ slow
- hot ↔ cold
- up ↔ down
- rise ↔ fall

Connotation (sentiment)

- Every word has two kinds of meaning:
  - 1. Literal meaning (dictionary meaning)
- What the word actually refers to.
  - 2. Connotation (emotional meaning)
- How the word feels the emotion, sentiment, or opinion associated with it.
- This is what “affective meaning” means.
- Connotation = Emotional Value of a Word
- Words have **affective** meanings
  - Positive connotations (*happy*)
  - Negative connotations (*sad*)
- Connotations can be subtle:
  - Positive connotation: *copy, replica, reproduction*
  - Negative connotation: *fake, knockoff, forgery*
- Evaluation (sentiment!)
  - Positive evaluation (*great*, *love*)
  - Negative evaluation (*terrible*, *hate*)

Connotation

- Early Work on Affective Meaning (Osgood et al., 1957)
- Researchers discovered that words don't just have dictionary meanings they also carry emotional meanings that vary along three dimensions.
- These dimensions describe how a word makes people feel.
- 1 Valence: How pleasant or unpleasant a word feels
  - Valence = emotional positivity or negativity.
  - High valence = pleasant (e.g., "happy", "love", "beautiful")
  - Low valence = unpleasant (e.g., "sad", "hate", "terrible")
- 2 Arousal: How intense the emotion is
- Arousal = how strong or weak the emotional reaction is.
  - High arousal = strong, exciting, activating
    a. (e.g., "explosion", "furious", "ecstatic")
  - Low arousal = calm, quiet, relaxing
    a. (e.g., "peaceful", "sleepy", "calm")

3. Dominance: How powerful or controlling the emotion feels

- Dominance = sense of control or lack of control related to the word.
  - High dominance = powerful, in control
  - (e.g., "hero", "command", "victory")
- Low dominance = powerless, controlled
  - (e.g., "fear", "terror", "prison")

Connotation

	Word	Score		Word	Score
Valence	love	1.000		toxic	0.008
	happy	1.000		nightmare	0.005
Arousal	elated	0.960		mellow	0.069
	frenzy	0.965		napping	0.046
Dominance	powerful	0.991		weak	0.045
	leadership	0.983		empty	0.081

So far

- **Concepts** or word senses
  - Have a complex many-to-many association with **words** (homonymy, multiple senses)
- Have relations with each other
  - Synonymy
  - Antonymy
  - Similarity
  - Relatedness
  - Connotation

So far

- **Concepts** or word senses
  - Have a complex many-to-many association with **words** (homonymy, multiple senses)
- Have relations with each other
  - Synonymy
  - Antonymy
  - Similarity
  - Relatedness
  - Connotation

Vector Semantics & Embeddings

Vector Semantics

Computational models of word meaning

- Can we build a theory of how to represent word meaning, that accounts for at least some of the desiderata?
- We'll introduce **vector semantics**
  - The standard model in language processing!
  - Handles many of our goals!
- Vector semantics is the standard way to represent word meaning in NLP, helping us model many of the aspects of word meaning we saw in the previous section.
- The roots of the model lie in the 1950s when two big ideas converged:

Ludwig Wittgenstein

- Philosophical Investigations, Section 43 a famous book by the philosopher Ludwig Wittgenstein (published in 1953).
- In this section, Wittgenstein gives one of the most important ideas in language and meaning:

"The meaning of a word is its use in the language"

- (PI §43) This means:
- To understand what a word means,
- look at how people actually use it.
- Meaning comes from context, usage, and behavior,
- not just a dictionary definition.

Why is PI 43 important for NLP?

- It directly supports vector semantics and word embeddings.
- Because if meaning = use, then:
- We can study meanings by looking at the contexts where words appear.
- This is exactly what modern NLP does:
  - Word2Vec
  - GloVe
  - FastText
  - BERT
- All learn meaning from usage patterns in text.

Let's define words by their usages

- In vector semantics, usage means:
- The words that appear around a target word (its environment or context).
- Words that occur in similar environments tend to have similar meanings.
- Zellig S. Harris (1909–1992) was a famous American linguist.
  - He worked at the University of Pennsylvania and was one of the most influential linguists of the 20th century.
- **Zellig Harris (1954): Distributional Hypothesis**
- Harris proposed:
  - If two words (A and B) occur in almost the same environments, then they are synonyms (or close in meaning).
- Example:
- “dog” appears near: bark, tail, pet
- “cat” appears near: meow, tail, pet
- Their environments are similar
- So their meanings are similar
- This is the foundation of word embeddings.

What does the word “ongchoi” mean

- But you see it in these sentences:
  - *Ong choi is delicious sautéed with garlic.*
  - *Ong choi is superb over rice.*
  - *Ong choi leaves with salty sauces.*
- And you also know these examples:
  - *Spinach sautéed with garlic over rice*
  - *Chard stems and leaves are delicious*
  - *Collard greens with salty leafy greens*
- **From these patterns, you can guess:**
- **Ong choi is a leafy green vegetable**, similar to:
  - spinach
  - chard
  - collard greens
- Why?
  Because the **context words** around “ong choi” are the same as the ones used with leafy greens:
- delicious
- sautéed
- leaves
- salty sauces
- over rice
- The **similar environment → similar meaning**.

Ongchoi: *Ipomoea aquatica "Water Spinach"*

空心菜
kangkong
rau muống
…
Here's a picture of ongchoi for you food fans

Yamaguchi, Wikimedia Commons, public domain

Idea 1: Defining meaning by linguistic distribution

- Let's define the meaning of a word by its distribution in language use, meaning its neighboring words or grammatical environments.
- So, we can express this first intuition as Idea 1:

Idea 2: Meaning as a point in space (Osgood et al. 1957)

- **Affective Dimensions → Vectors**
- **Words have 3 affective dimensions (Osgood, 1957):**
- **Valence** → How pleasant or unpleasant a word feels
- **Arousal** → How intense or calm the emotion is
- **Dominance** → How powerful or powerless the word feels
- These 3 numbers describe a word's **emotional meaning (connotation)**.
- Connotation as a Vector

- Because each word has 3 values, we can represent a word's connotation as a vector in 3-dimensional space.
- Example:
- Word → [Valence, Arousal, Dominance] = Vector
- happy → [8.5, 6.0, 7.2] =(8.5, 6.0, 7.2)
- fear → [2.0 , 8.8,1.5] = (2.0, 8.8, 1.5)
- So each word = a point in 3D space.
- This is the second big idea behind vector semantics.

	Word	Score	Word	Score
Valence	love	1.000	toxic	0.008
	happy	1.000	nightmare	0.005
Arousal	elated	0.960	mellow	0.069
	frenzy	0.965	napping	0.046
Dominance	powerful	0.991	weak	0.045
	leadership	0.983	empty	0.081

NRC VAD Lexicon (Mohammad 2018)

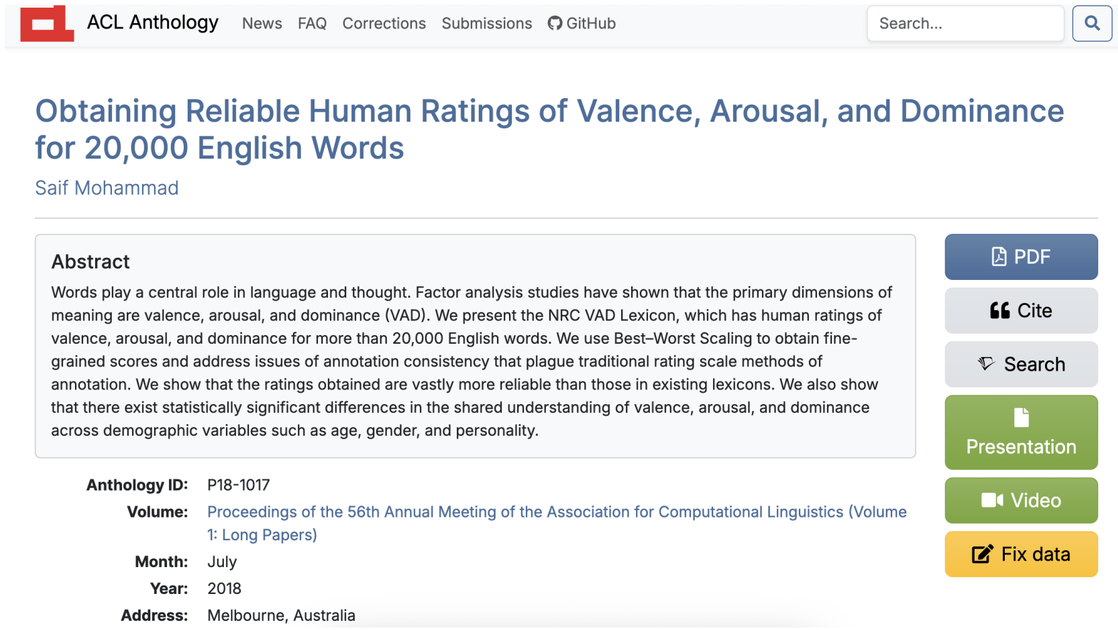
Must Read Part of the Exam

1. Obtaining Reliable Human Ratings of Valence, Arousal, and Dominance for 20,000 English Words

- https://aclanthology.org/P18-1017/
- https://aclanthology.org/P18-1017.mp4

2. NRC VAD Lexicon v2: Norms for Valence, Arousal, and Dominance for over 55k English Terms

- https://arxiv.org/abs/2503.23547



Idea 1: Defining meaning by linguistic distribution

Idea 2: Meaning as a point in multidimensional space

So we're going to combine these two ideas:

Vector Semantics: Defining Meaning as a Point in Space

Each word = a vector

- In vector semantics, a word is not just:
- A string like "good", or an ID like w_{45}
- Instead, it becomes a vector:
- Like good → [0.81, 0.12, –0.54, ...] (60-dimensional)
- This vector is its meaning.

Meaning = a point in semantic space

- Each vector places the word somewhere in a multi-dimensional space
- (20D, 60D, 300D, etc.).
- Words with similar meanings appear near each other in that space.
- Example:
  - happy → near joy, delight, cheerful
  - sad → near unhappy, depressed

Similar words = nearby in space

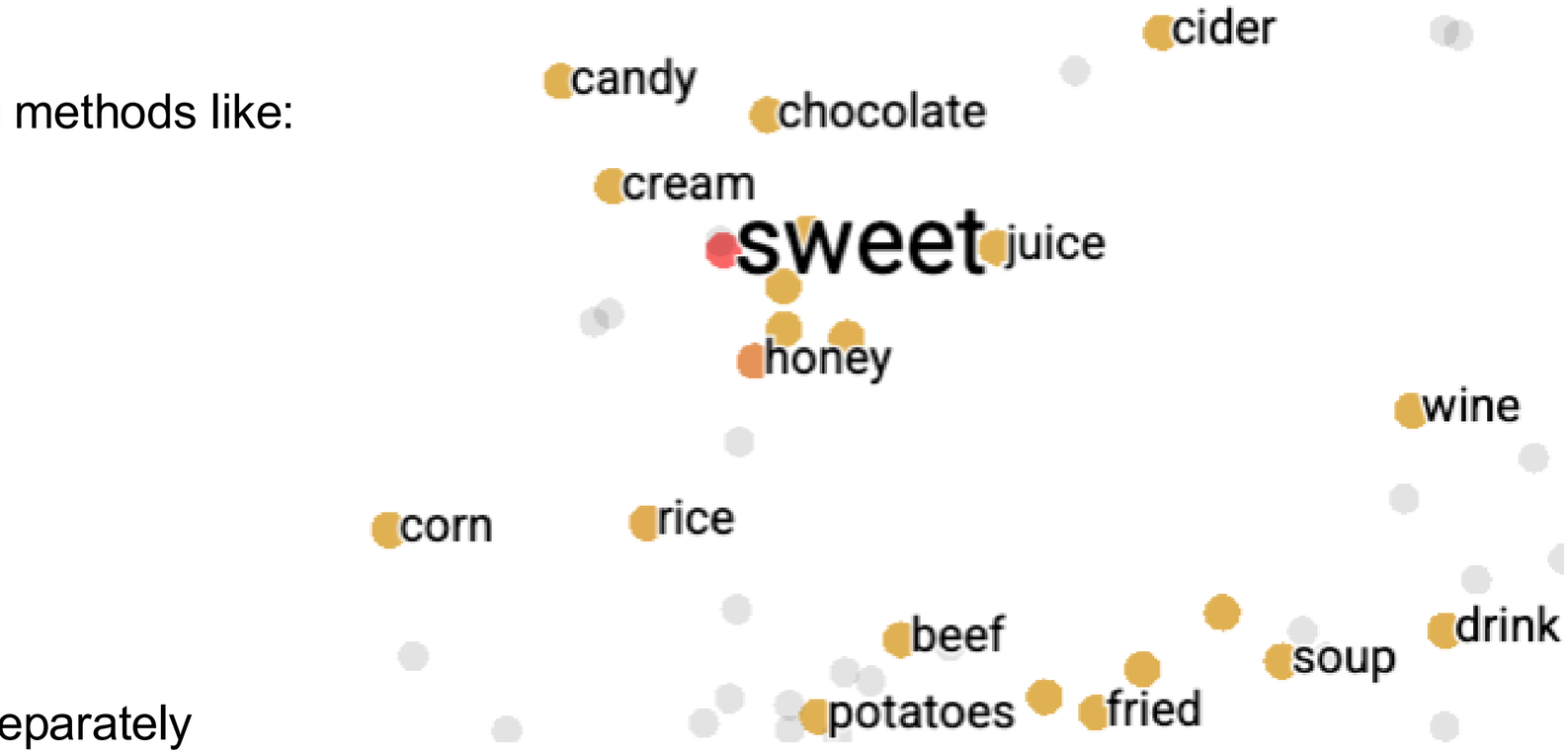
- If two words share contexts (appear with similar neighbors), their vectors become close.
- This captures:
  - synonymy
  - relatedness
  - sentiment polarity
  - topic similarity

How do we build this semantic space? Automatically.

- We scan a large corpus and count:
- Which words appear near which.
- This produces:
  - co-occurrence counts
  - context windows
  - distribution vectors
- Algorithms like Word2Vec or GloVe convert these counts into dense embeddings.

Visualizing high-dimensional meaning spaces

- Vector spaces usually have **dozens or hundreds** of dimensions.
- We **project** them down to **2D** using methods like:
  - PCA
  - t-SNE
  - UMAP
- Then you see neat clusters:
- **positive words** grouped together
- **negative words** grouped together
- **neutral function words** grouped separately
- This confirms that the model has captured **semantic structure**.



We define meaning of a word as a vector

- **Meaning as a Vector → Embedding**
- In NLP:
- Each **word = a vector (a list of numbers)**
- This vector is called an **embedding**
- Because the word is *embedded* into a multi-dimensional space (like 60D, 100D, 300D, etc.)
- This is the **standard representation** in modern NLP.

- Every neural NLP model uses embeddings:
  - Machine translation
  - Sentiment analysis
  - Chatbots
  - Search engines
  - Question answering
  - BERT, GPT, Word2Vec, GloVe

Intuition: Why vectors?

Why Vectors Instead of Strings?

- Think about **sentiment analysis**.

Old way:

- Feature = the exact word

  If your training data has:

- “terrible food”
- and in the test data the model sees:
- “horrible food”
- The model FAILS because:
- “terrible” ≠ “horrible”
- They are different strings → cannot generalize

New vector way:

- Feature = the word’s embedding:
- terrible → [35, 22, 17, …]
- horrible → [34, 21, 14, …]
- These vectors are **close** to each other.
- Meaning: Even if the model has never seen “horrible,”

  It will recognize it is similar to “terrible.”

  So the model generalizes better.
- This is the **power of embeddings**.

We'll Discuss 2 Kinds of Embeddings

- **How Do We Learn These Vectors Automatically?**
- Two main methods:

1. Simple Count Embeddings

- *(Classical / sparse vectors)*
- **Method:**
- Count which words appear near each word
- Create a **word × context** matrix
- Each word is represented by **counts** of neighbors
- Example: cat → [12 (near "pet"), 7 (near "tail"), 3 (near "food"), …]
- These vectors are big and **sparse**.
- **Used in:**
- early NLP
- tf-idf (information retrieval)

2. Word2Vec (Neural Embeddings)

- *(Modern / dense vectors)*
- **Method:**
- Train a small neural network:
- Given a word → predict its nearby words

  OR
- Given nearby words → predict the center word
- During this training:
- The model learns **dense, meaningful** vectors
- Similar words get similar vectors
- **Used in:**
- Word2Vec
- GloVe
- FastText
- Modern large models

Contextual Embeddings (Future / advanced)

- (BERT, GPT, etc.)
- Now words don’t have one fixed vector.

  They have **different vectors depending on context.**
- Example: bank in:
- *river bank*
- *bank account*
- Two different vectors!

Vector Semantics & Embeddings

Count-based embeddings

Let's talk about how to represent words as simple vectors of context counts

Remember the intuition: words with similar neighborhoods have similar meanings

How to measure a word's neighborhood?

Word-context matrix (a kind of co-occurrence matrix)

- each row represents a word in the vocabulary
- each column represents how often each other word in the vocabulary appears nearby

Word–Context Matrix

- **How Do We Measure a Word's "Neighborhood"?**
- To know what a word *means*, we look at the words that appear **near it** in text.
- This is based on the **Distributional Hypothesis**:
- **Words that appear in similar neighborhoods have similar meanings.**
- A **word–context matrix** is a big table where:
- **Each row = a word in the vocabulary**
- **Each column = a context word**
- **Each cell = how often the context word appears near the target word**
- This is also called a **co-occurrence matrix**.
- **Example**
- Imagine a tiny corpus:

- cat sat on the mat
  dog sat on the rug
  the cat chased the dog
- We build a matrix of counts (window size = 1 word):
- **What this tells us:**
- "cat" and "dog" appear with similar neighbors: **the, sat, on**
- So they get **similar rows → similar vectors → similar meaning**
- This is how we measure the **semantic neighborhood**.

Word–Context Matrix

- We use 2 simple sentences:
- 1. the cat ate fresh fish
- 2. the dog ate tasty meat
- Window size = **2**
  take **2 words left** and **2 words right** of the target word.
- We build vectors for **cat** and **dog**.
- Vocabulary (context words):
- [the, cat, dog, ate, fresh, tasty, fish, meat]

Step 1 — Build vector for "cat" (window = 2)

- Sentence: the cat ate fresh fish
- Word positions:
- 1: the
- 2: cat
- 3: ate
- 4: fresh
- 5: fish
- Target = **cat** at position **2**
- **Words within window size = 2:**
- Left side: pos 1 → "the"
- Right side: pos 3 → "ate" , pos 4 → "fresh"
- pos 5 ("fish") is 3 words away → too far.
- **So the context words for "cat" are:**
- the, ate, fresh
- **Count in vocabulary order:**
- Vocabulary order:
- [the, cat, dog, ate, fresh, tasty, fish, meat]
- Vector for **cat**:
- cat → [1, 0, 0, 1, 1, 0, 0, 0]
- Explanation:
- "the" → 1
- "cat" → 0
- "dog" → 0
- "ate" → 1
- "fresh" → 1
- "tasty" → 0
- "fish" → 0
- "meat" → 0

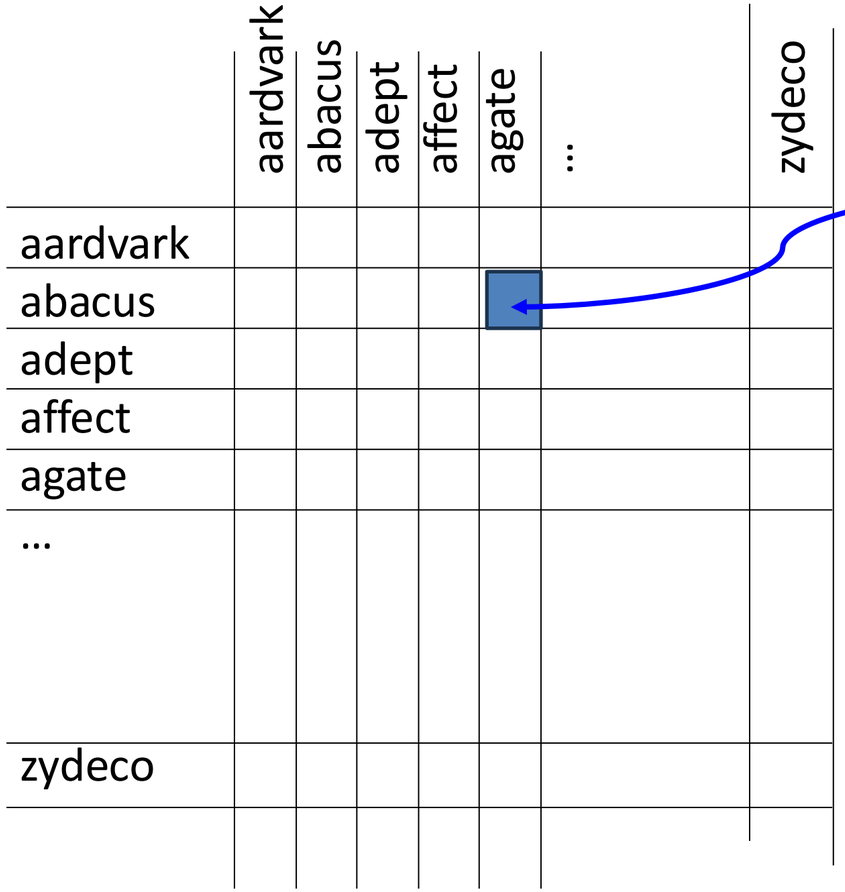
Step 2 — Build vector for "dog" (window = 2)

- Sentence:
- the dog ate tasty meat
- Word positions:
- 1: the
- 2: dog
- 3: ate
- 4: tasty
- 5: meat
- Target = **dog** at position **2**
- **Words within window size = 2:**
- Left side:
- pos 1 → "the"
- Right side:
- pos 3 → "ate"
- pos 4 → "tasty"
- pos 5 ("meat") is 3 words away → too far.
- **So the context words for "dog" are:**
- the, ate, tasty
- **Count in vocabulary order:**
- Vocabulary:
- [the, cat, dog, ate, fresh, tasty, fish, meat]
- Vector for **dog**:
- dog → [1, 0, 0, 1, 0, 1, 0, 0]
- Explanation:
- "the" → 1
- "cat" → 0
- "dog" → 0
- "ate" → 1
- "fresh" → 0
- "tasty" → 1
- "fish" → 0
- "meat" → 0

Final Result (Window = 2)

- cat → [1, 0, 0, 1, 1, 0, 0, 0]
- dog → [1, 0, 0, 1, 0, 1, 0, 0]
- Both share **the** and **ate** → vectors partly similar
- Cat has **fresh**
- Dog has **tasty**
- This shows: 👉 Similar contexts → similar vectors
  👉 Different contexts → different vector components
- This is the **distributional hypothesis**.

Word-Context Matrix



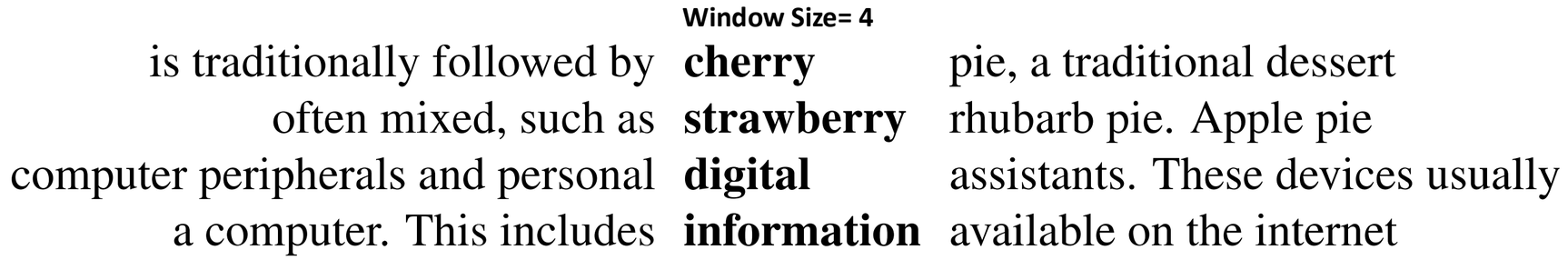
How often does **agate** occur near **abacus**?

Word-Context Matrix

- What does "nearby" mean?
- For right now let's say "within 4 words"

The word-context matrix

- A word-context matrix is a big table where:
  - Each row = a target word
  - Each column = a context word
  - Each cell = number of times the context word appears near the target word
- This is also called:
  - term–term matrix
  - word–word matrix
  - term–context matrix
  - co-occurrence matrix
- All of them mean the same idea.



The word-context mini-matrix for just 4 words and 3 contexts

is traditionally followed by **cherry** pie, a traditional dessert
often mixed, such as **strawberry** rhubarb pie. Apple pie
computer peripherals and personal **digital** assistants. These devices usually
a computer. This includes **information** available on the internet

	a	computer	pie
cherry	1	0	1
strawberry	0	0	2
digital	0	1	0
information	1	1	0

The word-context mini-matrix for just 4 words and 3 contexts

- This 4x3 matrix is a subset of full |V| x |V| matrix
- Each word is represented by a row vector with dimensionality [1 x |V|]
- With co-occurrence counts with each other word

	a	computer	pie
cherry	1	0	1
strawberry	0	0	2
digital	0	1	0
information	1	1	0

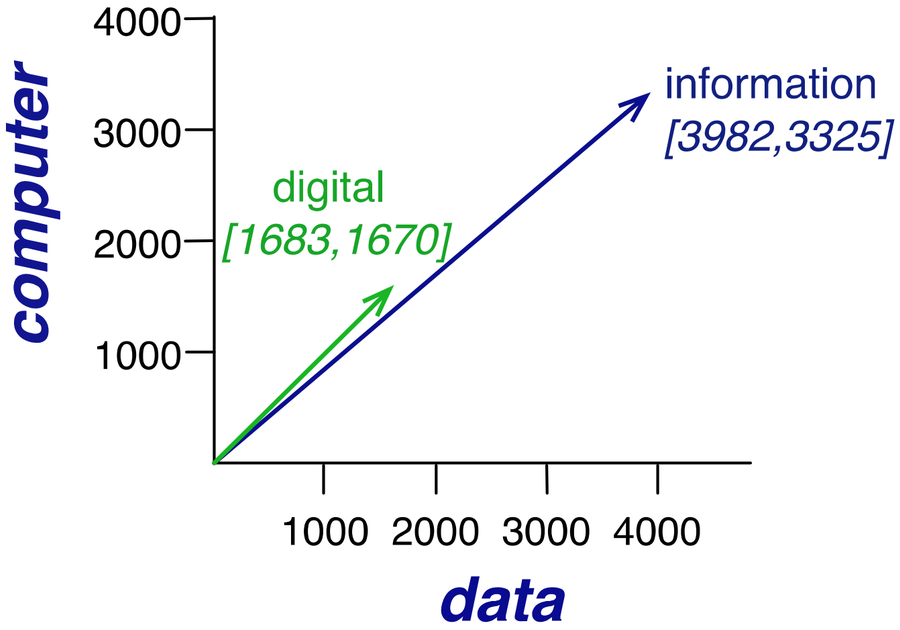
A selection from a larger word-context matrix

is traditionally followed by	**cherry**	pie, a traditional dessert
often mixed, such as	**strawberry**	rhubarb pie. Apple pie
computer peripherals and personal	**digital**	assistants. These devices usually
a computer. This includes	**information**	available on the internet

	aardvark	...	**computer**	**data**	**result**	**pie**	**sugar**	...
cherry	0	...	2	8	9	442	25	...
strawberry	0	...	0	0	1	60	19	...
digital	0	...	1670	1683	85	5	4	...
information	0	...	3325	3982	378	5	13	...

A selection from a larger word-context matrix

Here's a spatial visualization of word vectors for digital and information, showing just two of the dimensions, corresponding to the words data and computer.



- This picture shows only 2 dimensions of two word vectors:
  - the data dimension (x-axis)
  - the computer dimension (y-axis)
- So each word is shown as a point:
  - digital → (1683, 1670)
  - information → (3982, 3325)
  - “digital” appears 1683 times near the word data
  - “digital” appears 1670 times near the word computer
  - “information” appears 3982 times near data
  - “information” appears 3325 times near computer
- These numbers come from how often the word appears near data and computer in a corpus.

In reality, a word vector is not 2-dimensional.

- The real vector has |V| dimensions, where |V| is the number of words in your vocabulary.
- If your vocabulary has 10,000 words → each word’s vector has 10,000 numbers.
- If vocabulary has 50,000 words → each vector has 50,000 numbers.
- That’s why the vector is huge.

The word-context matrix

- Word context matrix is |V| x |V|
- This could be 50,000 x 50,000

  Most of these numbers are zero!
- So these are sparse vectors
- There are efficient algorithms for storing and computing with sparse matrices

Vector Semantics & Embeddings

Count-based Embeddings

Cosine for Computing Word Similarity

To measure similarity between two words, we need a metric that compares two vectors.

By far the most common similarity metric is the cosine of the angle between the vectors.

Computing word similarity

- **What is dot product?**
- Given two word vectors **v** and **w**:
- $dot\ product(v, w) = \sum_{i=1}^{N} v_i\, w_i$

$$\text{dot product}(\mathbf{v}, \mathbf{w}) = \mathbf{v} \cdot \mathbf{w} = \sum_{i=1}^{N} v_i w_i = v_1 w_1 + v_2 w_2 + \ldots + v_N w_N$$

- Multiply each dimension

  Then add up all the multiplications
- Example with 3-dimensional vectors:
- v = [2, 5, 1], w = [3, 4, 0]
- Dot product:
- 2×3 + 5×4 + 1×0 = 6 + 20 + 0 = 26
- The result is **one number** (a scalar).

Why does dot product measure similarity?

- Dot product becomes **large** when:
  - both vectors have **large values in the SAME dimensions**
  - both vectors point in a **similar direction**
- Example:
- If two word vectors have:
- v = [big, small, big, small]
- w = [big, small, big, small]
- Multiplying big×big gives big numbers
- Multiplying small×small gives small numbers
- Result = large → meaning **vectors are similar**.
- But if the words are unrelated:
- v = [big, small, big, small]
- w = [0, 0, 0, 0]
- Dot product = very small

  → words are **not similar**

Intuition with word meaning

- If two words appear with similar context words:
  - **digital** appears with **computer**
  - **information** appears with **computer**
- Then both vectors have:
  - computer dimension = large
  - data dimension = large

$$digital \cdot \textbf{information}$$

- is **big**.
- But:
- **strawberry** does NOT appear with data or **computer**

  → its vector values for these dimensions are **0 or small**
- **So:**

$$digital \cdot \textbf{strawberry}$$

- is **small**.
- Therefore:
- Dot product captures **semantic similarity**

Dot Product vs Cosine

Dot product checks:

- “Do these vectors have large numbers in the same places?”

Cosine similarity:

- “Do these vectors point in the same direction, regardless of length?”
- Dot product is the **simpler** metric.

  One-line summary
- **Dot product is high when two word vectors have large values in the same dimensions**
  - Which means they appear in similar contexts
  - So it is useful for measuring semantic similarity.

Long vs Short Vector

- A **long vector** is a vector with **big numbers** in many dimensions.
- v = [50, 40, 10, 80]
- This is a **long** (large-magnitude) vector.
- A short vector: w = [1, 0, 2, 1]
- Even if the words are not similar,
  the dot product will be **much higher** for the long vector.
- **Why are frequent words long vectors?**
- Words like:
  - **the**
  - **of**
  - **you**
  - **and**
- appear in **thousands** of contexts.
- So their co-occurrence counts are huge:
- **the** → [5000, 4000, 3000, 2000, ...]
- This creates a very **long vector**.
- Meanwhile, a rare word like **zebra**:
- zebra → [1, 0, 2, 0, ...]
- is a **short vector**.

Problem with Raw Dot Product

- **Why dot product favors long vectors**
- Dot product = multiply and sum:
  - $v \cdot w = \sum_i v_i w_i$
- If a vector has **big numbers in many dimensions**,
  the dot product will naturally be high.
- **dot(the, zebra)** → huge (because "the" vector is huge)
- **dot(dog, cat)** → smaller (because these vectors are normal-sized)
- This is wrong because:
- "the" is NOT more similar to "zebra" **than** "dog" is to "cat"
- But dot product says it IS,
  because "the" has huge counts.
- The slide says:
- **The vector length is defined as the square root of the sum of the squares**
- This is the standard definition:
- $\| v \| = \sqrt{v_1^2 + v_2^2 + \cdots + v_n^2}$
- A frequent word → large entries → large length

$$|\mathbf{v}| = \sqrt{\sum_{i=1}^{N} v_i^2}$$

Problem with Raw Dot Product

- **Why this is a problem**
- Dot product becomes high for **frequent words**, even when meanings are unrelated.
- For example:
- dot(the, strawberry) = large (wrong)
- dot(digital, information) = medium
- The dot product incorrectly says:
- “the” is more similar to “strawberry” than “digital” is to “information”
- because **“the”** is extremely frequent.
- This is why:
- 🔥 **Raw dot product is NOT a good similarity metric for word meaning.**

Alternative: Cosine Similarity

- **Cosine Similarity:** $cosine(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{\|\vec{v}\| \ \|\vec{w}\|}$
- **Top part (dot product):**
  - How much the vectors align
- **Bottom part (product of lengths):**
  - Removes the effect of frequency / size
- So cosine similarity measures:
- **How close the direction of two vectors is**, regardless of their length.

$$\text{cosine}(\vec{v}, \vec{w}) = \frac{\vec{v} \cdot \vec{w}}{|\vec{v}||\vec{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2} \sqrt{\sum_{i=1}^{N} w_i^2}}$$

Alternative: Cosine Similarity

- **This normalized dot product is the cosine of the angle between the two vectors.**
- This comes from geometry:
- In geometry:
- $\vec{a} \cdot \vec{b} = \| \vec{a} \| \quad \| \vec{b} \| \cos\theta$
- If you divide both sides by $\| \vec{a} \| \| \vec{b} \|$:
- $\frac{\vec{a} \cdot \vec{b}}{\|\vec{a}\|\|\vec{b}\|} = \cos\theta$
- So cosine similarity is literally:
- **cosine(θ)**

  where θ = angle between the two word vectors.

$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}||\mathbf{b}| \cos\theta$$

$$\frac{\mathbf{a} \cdot \mathbf{b}}{|\mathbf{a}||\mathbf{b}|} = \cos\theta$$

Alternative: Cosine Similarity

- **Why is angle a good similarity measure?**
- Because:
- If θ = 0° → vectors point in the **same direction** → **cosine = 1**
- words have very similar meaning
- If θ = 90° → vectors are **unrelated** → **cosine = 0**
- words have no shared meaning
- If θ = 180° → vectors point opposite → **cosine = –1**
- very different meanings
- So cosine captures **semantic similarity**, not frequency.
- **Simple Example**
- Imagine two word vectors:
- digital → [3, 4]
- information → [6, 8]
- **Dot product:**
- $3 \cdot 6 + 4 \cdot 8 = 18 + 32 = 50$
- **Vector lengths:**
- $\| \, digital \, \| = \sqrt{3^2 + 4^2} = 5$
- $\| \, information \, \| = \sqrt{6^2 + 8^2} = 10$
- Cosine similarity:
- $\frac{50}{5 \cdot 10} = 1$
- Meaning:
- **Both vectors point in exactly the same direction**
- **Words are extremely similar**

Why cosine solves the dot-product problem

- Dot product was unfair because:
- frequent words → long vectors → bigger dot product
- even if meanings were unrelated
- Cosine **fixes** this by dividing by vector length.
- So cosine asks:

  “Are these vectors pointing in the same direction?”

  NOT

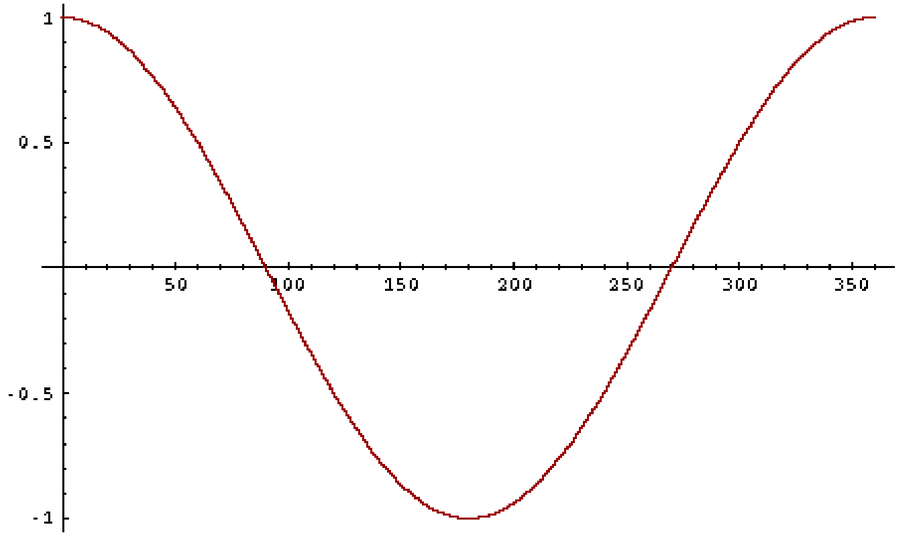
  “Are these vectors long?”

- This gives a **better measure of word meaning**.

Cosine as a similarity metric

- -1: vectors point in opposite directions
- +1: vectors point in same directions
- 0: vectors are orthogonal, “Orthogonal” means at 90° angle = completely unrelated.
- The cosine value ranges from 1 for vectors pointing in the same direction, through 0 for orthogonal vectors, to -1 for vectors pointing in opposite directions.
- But since raw frequency values are non-negative, the cosine for these vectors ranges from 0–1.

Cosine value	Meaning	NLP word example
+1	same direction → identical meaning	(water, water), (dog, dogs)
~0.8–1	very similar	(digital, information), (happy, joyful)
0	orthogonal → unrelated	(banana, democracy)
–1	opposite direction → opposite meaning	Does NOT happen with frequency vectors



Cosine Examples

$$\cos(\vec{v},\vec{w}) = \frac{\vec{v}\bullet\vec{w}}{|\vec{v}||\vec{w}|} = \frac{\vec{v}}{|\vec{v}|}\bullet\frac{\vec{w}}{|\vec{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$

	pie	data	computer
cherry	442	8	2
digital	5	1683	1670
information	5	3982	3325

Cosine Examples

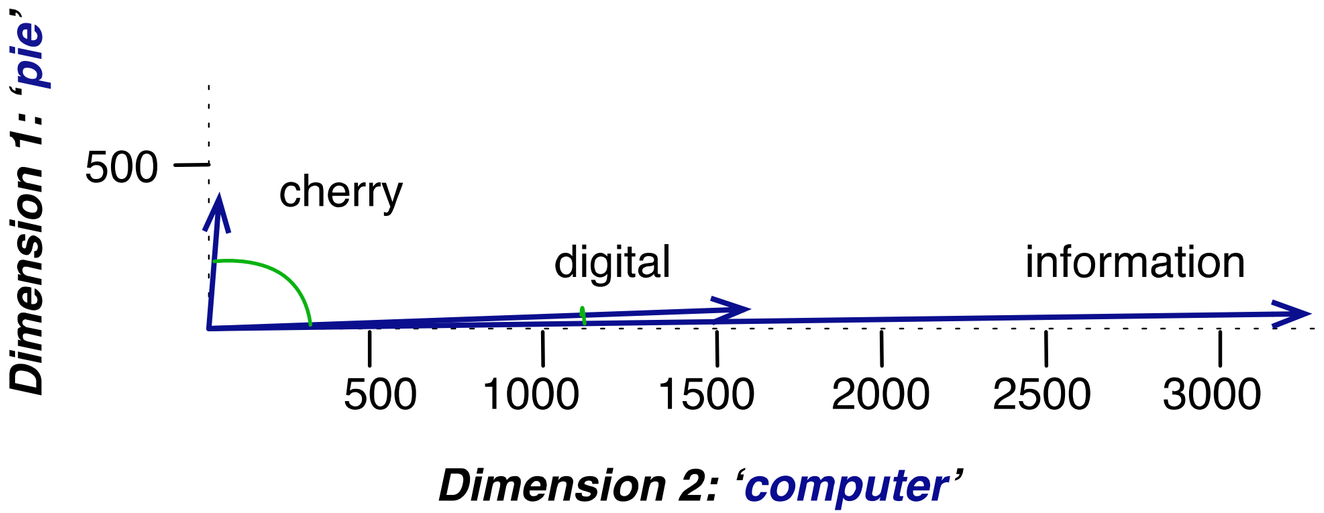
$$\cos(\vec{v},\vec{w}) = \frac{\vec{v}\bullet\vec{w}}{|\vec{v}||\vec{w}|} = \frac{\vec{v}}{|\vec{v}|}\bullet\frac{\vec{w}}{|\vec{w}|} = \frac{\sum_{i=1}^{N} v_i w_i}{\sqrt{\sum_{i=1}^{N} v_i^2}\sqrt{\sum_{i=1}^{N} w_i^2}}$$

$$\cos(\text{cherry},\text{information}) = \frac{442*5+8*3982+2*3325}{\sqrt{442^2+8^2+2^2}\sqrt{5^2+3982^2+3325^2}} = .017$$

$$\cos(\text{digital},\text{information}) = \frac{5*5+1683*3982+1670*3325}{\sqrt{5^2+1683^2+1670^2}\sqrt{5^2+3982^2+3325^2}} = .996$$

Visualizing cosines (well, angles)

- Here's a rough graphical demonstration of cosine similarity, showing vectors for the words *cherry*, *digital*, and *information* in the two dimensional space defined by counts of the words *computer* and *pie* nearby.
- Note that the angle between *digital* and *information* is smaller than the angle between *cherry* and *information*.
- When two vectors are more similar, the cosine is larger but the angle is smaller; the cosine has its maximum (1) when the angle between two vectors is smallest (0◦); the cosine of all other angles is less than 1.



Vector Semantics & Embeddings

Word2vec

Let's now Introduce the Word2vec Embeddings

Sparse vs Dense Vectors

Sparse, Count-Based Vectors (Old Method)

- These come from:
  - co-occurrence counts
  - tf-idf (term-frequency / inverse-document-frequency)
  - word–context matrices
- **These vectors are Long**
- Their length = vocabulary size |V|
  Usually: |V| = 20,000 to 50,000 words
- So each word becomes a vector with **20k–50k dimensions**.
- **Example:**
- cat → [0, 12, 0, 0, 5, 0, 0, ... 0] (length ~50,000)

Sparse

- Most values are **zero**.
- **Why?** Because a word appears with **very few** of the 20,000 context words.

Example:

- out of 50,000 dimensions, maybe 10 are non-zero
- Sparse = mostly zeros, only a few non-zero values.

Sparse vs Dense Vectors

Dense, Learned Embeddings (Modern Method)

- These are learned by:
  - Word2Vec
  - GloVe
  - FastText
  - BERT (contextual)
- **These vectors are Short**
- Dimensions: d = 50 to 1000
- Much smaller than 20,000–50,000.
- Example:
- cat → [0.23, -0.87, 1.43, 0.02, ...] (length 100)

Dense

- Most values are **non-zero**.

Example:

- [0.23, -0.87, 1.43, 0.02, -0.44, 0.91, ...]
- Dense = almost all the vector positions contain **useful values**.
- **Values are real numbers**
- Not counts.
  - Values can be **positive or negative**
  - They represent abstract “meaning dimensions”
  - Each dimension does **not** correspond to a specific word
- These dimensions do **not** have human meanings like:
- dimension 1 = “fruitness”
- dimension 2 = “animallike”
- dimension 3 = “metal”
- They are learned features, not interpretable.

Why Dense Embeddings Are Better

- **They are short → fast computation**
- **They capture semantic meaning**
- Words with similar meaning get similar vectors.
- **They are dense → every dimension contains information**
- Not just empty zeros.
- **Real numbers → more expressive than raw counts**
- Allows similarities, analogies, clustering.

Why Dense Vectors?

- The embeddings used in Word2Vec, GloVe, FastText, BERT work **better than sparse count vectors** for many reasons.

1. Dense vectors are short → easier for machine learning

- Sparse vectors have length:
- |V| = 20,000 to 50,000 dimensions
- That means a classifier (like logistic regression, SVM, neural network) must learn:
- **20,000 to 50,000 weights per word**
- This is HUGE → slow, and easily overfits.
- Dense vectors have:
  - d = 50 to 1000 dimensions
- Much smaller → machine learning has **fewer parameters to tune** → generalizes better and avoids overfitting.

Why Dense Vectors?

2. Dense vectors generalize better than explicit counts

- Sparse count vectors only know:
  - exact counts of context words
  - exact co-occurrences
- But generalization requires understanding patterns, not exact matches.
- Dense embeddings learn:
  - smooth patterns
  - similarity
  - hidden abstract features
- They can understand *new* unseen contexts better.

Why Dense Vectors?

3. Dense vectors capture synonymy but sparse cannot

- Sparse vectors have **one dimension per word**:
  - Dimension 1 → car
  - Dimension 2 → automobile
- Even though *car* and *automobile* mean the same thing, they are in **separate dimensions** → unrelated in sparse vectors.
- That causes problems:
  - A word near “car”
  - and a word near “automobile”
- will look **different** in sparse vectors (because they count different dimensions).

Dense vectors fix this:

- Word2Vec learns that:
  - car ↔ automobile
- They end up **near each other** in the dense space.
- So:
  - words with “car”
  - words with “automobile”
  - become **similar**.
- This solves **synonymy**.

Common Methods for Short Dense Vectors

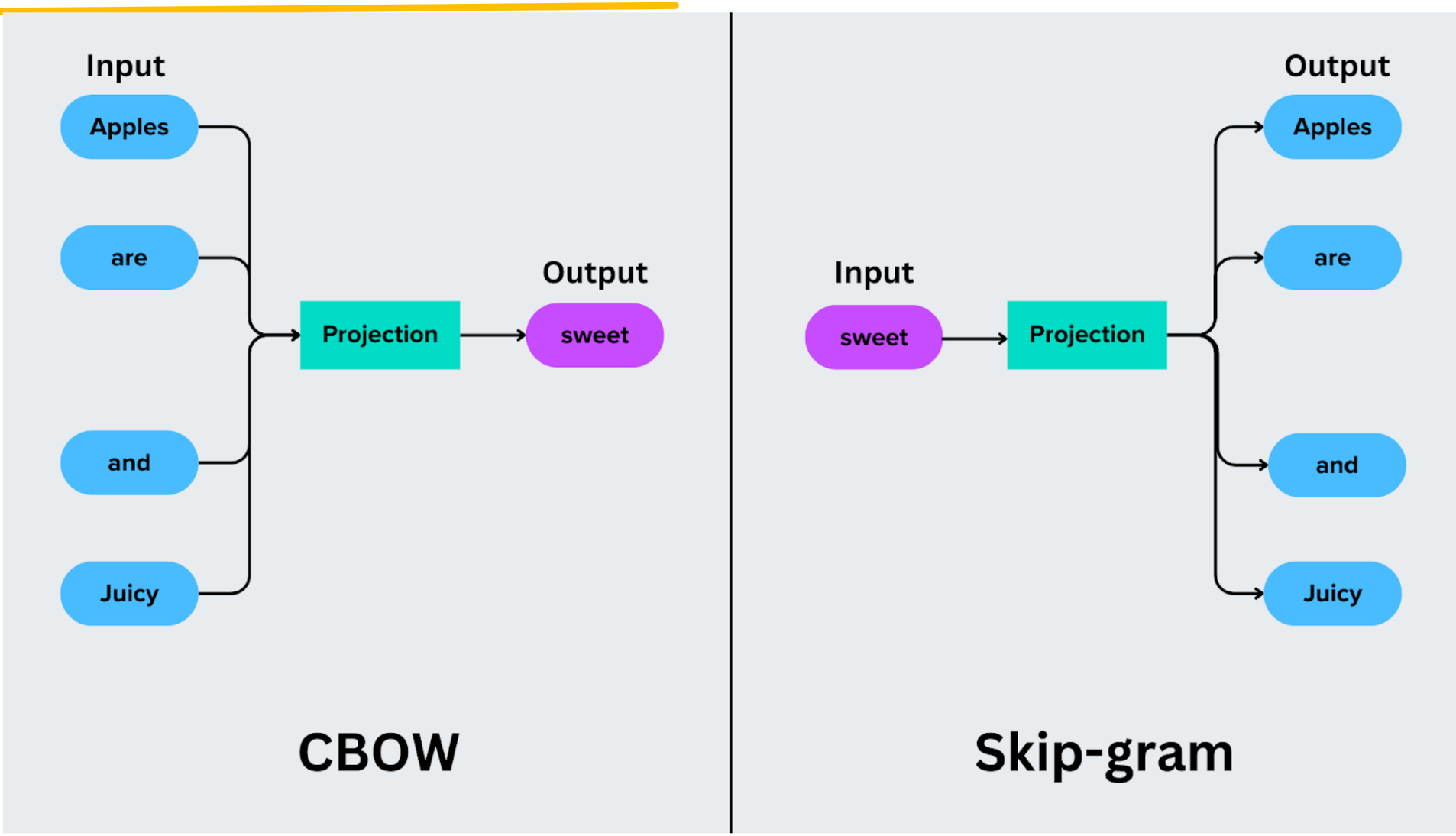
1. Neural Language Model (Inspired Methods)

- These methods learn **dense embeddings** by predicting words from context.
- **Word2Vec**
- Two algorithms:
  - **Skip-Gram (Vanialla or SGNS)**
  - **CBOW (Continuous Bag of Words)**
- **Word2Vec learns:**
  - **one dense vector per word**
  - vectors are **static** (same for every usage of the word)
  - values can be **positive or negative**

GloVe (Global Vectors)

- Uses matrix factorization + co-occurrence counts
- Also produces **dense**, **static embeddings**

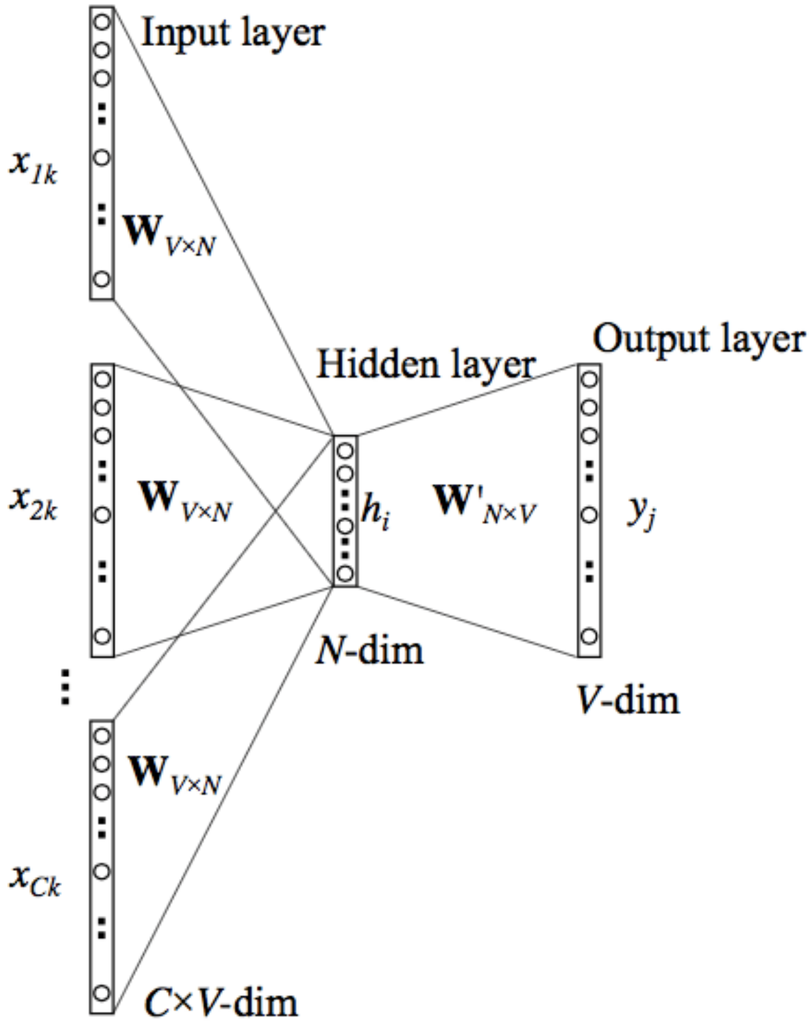
Word2Vec Two Algorithim

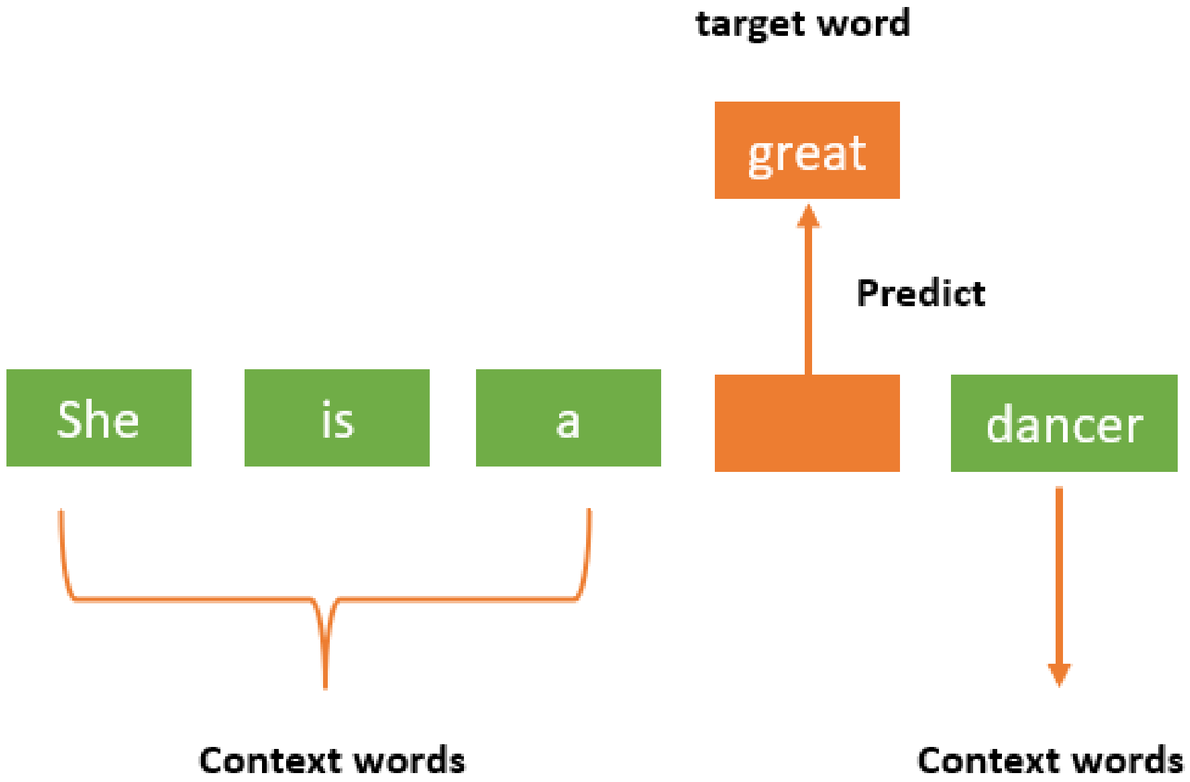


CBOW

- **1. CBOW (Continuous Bag of Words)**
- **Input = context words**
- **Output = target word**
- CBOW tries to **predict the center word** from the words around it.
- **Example:**
  - **the apricot tastes sweet** today
  - Window = ±2
- Context for target **“apricot”** =
  - [the, tastes, sweet]
- CBOW asks:
- **Given (the, tastes, sweet), what is the center word?**
- It tries to predict:
  - apricot
- **Context → Target**

CBOW


Input layer
Hidden layer
Output layer
x_{1k}
x_{2k}
x_{Ck}
$\mathbf{W}_{V\times N}$
$\mathbf{W}_{V\times N}$
$\mathbf{W}_{V\times N}$
h_i
$\mathbf{W}'_{N\times V}$
y_j
N-dim
V-dim
$C\times V$-dim

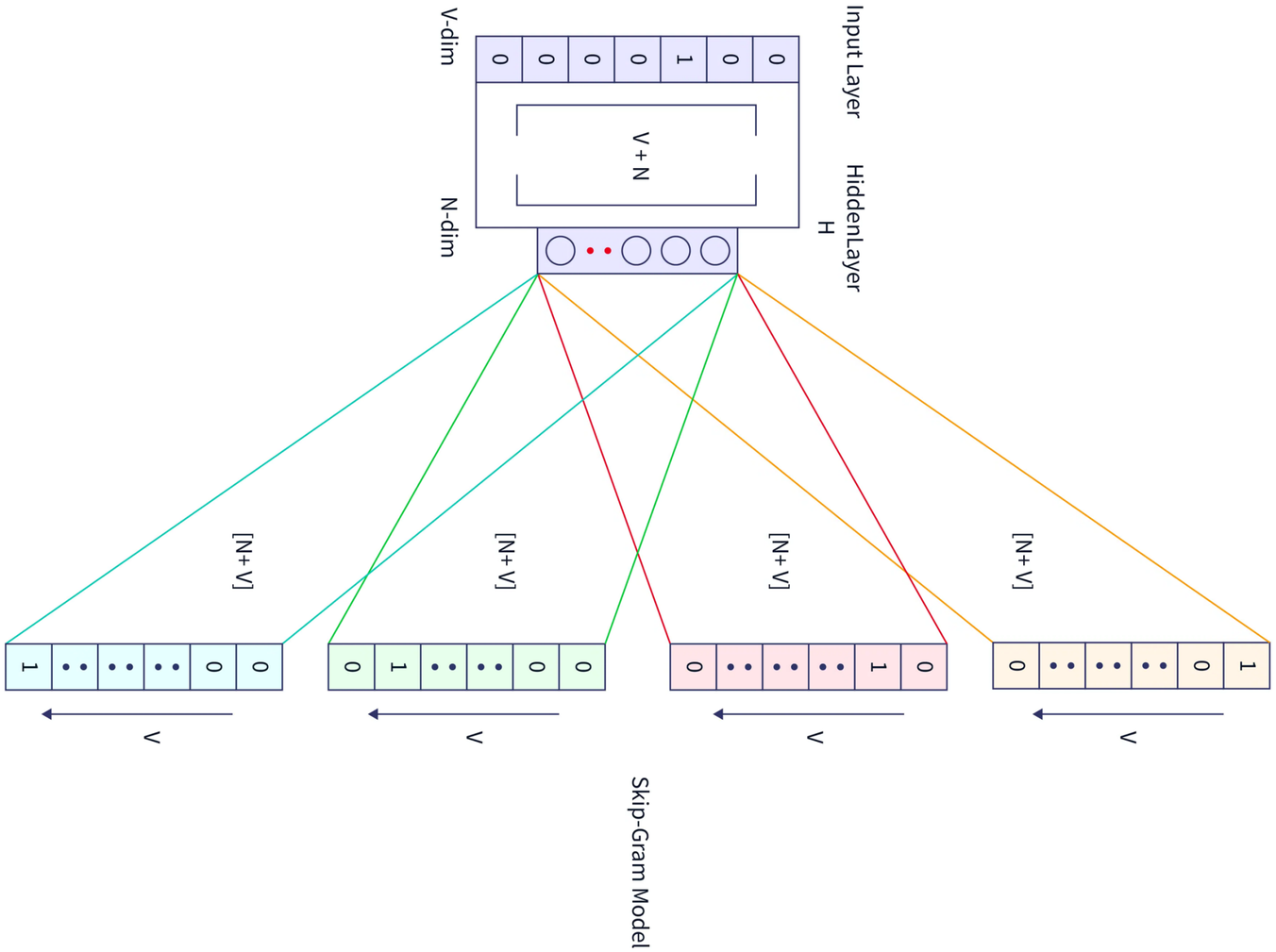

target word
great
Predict
She
is
a
dancer
Context words
Context words

Skip-Gram (SG or SGNS)

- **Input = target word**
- **Output = context words**
- Skip-gram does the **opposite** of CBOW.
- **Example:**
  - **the apricot tastes sweet today**
- Target = **apricot**
- Skip-gram asks:
- **Given the center word “apricot”, which words appear around it?**

- It tries to predict:
  - tastes
  - sweet
  - the
  - today
- **Target → Context**

Skip-Gram (SG or SGNS)



Common Methods for Short Dense Vectors

2. SVD (Singular Value Decomposition)

- This is a **mathematical method** that compresses a huge sparse matrix (like a word–context matrix) into a **smaller dense matrix**.
- A special case is:
- **LSA (Latent Semantic Analysis)**
- Uses SVD on a term-document matrix
  - produces **dense word vectors**
- This was used before neural embeddings became popular.

Static Embeddings (Word2Vec, GloVe)

These produce:

- **one vector per word**
- always the same, no matter the sentence
- Same embedding for **“bank”** in:
  - *river bank*
  - *bank account*
- So they cannot handle **polysemy** (multiple meanings of a word).

Contextual Embeddings (ELMo, BERT)

These are newer and much more powerful.

- **They compute a different vector for each token in context.**
- **Example:**
- Word **“bank”** will have two different embeddings:
  - The bank of the river → river meaning
  - I deposited money in the bank → financial meaning
- So BERT gives embeddings that reflect **contextual meaning**.

To summarize:

- **Static embeddings** = one vector for each word
- **Contextual embeddings** = different vector for each usage

Simple Static Embeddings You Can Download!

- Word2vec (Mikolov et al)
- https://code.google.com/archive/p/word2vec/

- GloVe (Pennington, Socher, Manning)
- http://nlp.stanford.edu/projects/glove/

Word2Vec

- **Main Idea: Predict Rather Than Count**
- Old method:
  - Count how often words appear near each other
    (e.g., how many times w appears near “apricot”)
- Word2Vec method:
  - Instead of counting, **predict**:
- **Is word w likely to appear near “apricot”?**
  (Yes or No → binary classification)
- This is called **self-supervised learning**.
  - Self-supervised learning (SSL) is a machine learning paradigm that enables models to learn from massive amounts of unlabeled data.
  - By creating their own "pseudo-labels," bridging the gap between supervised and unsupervised learning

Skip-Gram with Negative Sampling (SGNS)

- This is the specific Word2Vec algorithm we study.
- **SGNS does this:**
- For every target word (like **apricot**):
- **Positive examples** (words actually near apricot)
  - garlic
  - sweet
  - tree
  - fruit

    (these come from real text windows)
- **Negative examples** (random words NOT near apricot)
  - airplane
  - football
  - democracy
  - laptop

Task:

- Train a classifier to predict:

  Is this context word real (positive) or random (negative)?
- But here is the key:
- **We don't care about the classifier!**
- We care about the **weights** it learns

  those weights ARE the word embeddings.

SGNS: The Core Idea

- The approach is: **“Predict if a candidate word c is a neighbor of a target word t.”**
- Word2Vec turns the distributional hypothesis (Slide 26) into a **binary classification task**.
- Let’s break it down in the simplest way.

1. Treat real neighbors as *positive examples*

- Suppose your corpus contains:
  - apricot tastes sweet in summer
- Target word = **apricot**
- Words within a window of ±4 are:
  - tastes, sweet, in, summer
- These are **real neighbors → positive labels (1)**
- So we create training examples like:
  - (apricot, tastes) → 1
  - (apricot, sweet) → 1
  - (apricot, summer) → 1

2. Sample random words as *negative examples*

- Pick random words from the vocabulary:
  - airplane, football, democracy
- These are **fake neighbors → negative labels (0)**
  - (apricot, airplane) → 0
  - (apricot, democracy) → 0
- This is called **negative sampling**.

3. Train a logistic regression classifier

- For each pair (t, c):
  - **Is c a real neighbor of t?**
  - Output: 1 (yes) or 0 (no)
- This is just **binary classification** using logistic regression.
  - **Input:** word vectors (initially random)
  - **Output:** probability of being a neighbor
- We train the model on millions of such pairs.

Note: Words outside the window are **not positive** examples, but they are not automatically negative examples, **negative samples** are chosen randomly.

Why Word2Vec Was Revolutionary

- Earlier work (Bengio 2003, Collobert 2011) showed:
- You can train a neural model to predict the next word
- And learn good word embeddings from that prediction task
- But those models were **slow and complicated**
- Word2Vec simplified everything:
- **Simplifies the task**
- Instead of predicting the exact next word (huge softmax), Word2Vec turns it into a **binary yes/no problem**.
- **Simplifies the model**
- Uses **logistic regression**, not a deep neural network.
- The result:
  - MUCH faster
  - Much lighter
  - Easy to train on billions of words
  - Embeddings become very high-quality

Why Word2Vec Was Revolutionary

- Earlier work (Bengio 2003, Collobert 2011) showed:
- You can train a neural model to predict the next word
- And learn good word embeddings from that prediction task
- But those models were **slow and complicated**
- Word2Vec simplified everything:
- **Simplifies the task**
- Instead of predicting the exact next word (huge softmax), Word2Vec turns it into a **binary yes/no problem**.
- **Simplifies the model**
- Uses **logistic regression**, not a deep neural network.
- The result:
  - MUCH faster
  - Much lighter
  - Easy to train on billions of words
  - Embeddings become very high-quality

Skip-Gram Training Data

- Assume a +/- 2 word window, given training sentence:

[target]

- …lemon, a [tablespoon of apricot jam, a] pinch…

c1 c2 c3 c4

- Let's start by thinking about the classification task, and then turn to how to train.
- Imagine a sentence like the following, with a target word apricot, and assume we're using a window of ±2 context words.

Skip-Gram Classifier

- (assuming a +/- 2 word window)
- …lemon, a [tablespoon of apricot jam, a] pinch…
- c1 c2 [target] c3 c4
- Goal: train a classifier that is given a candidate (**w**ord, **c**ontext) pair
- (apricot, jam)
- (apricot, aardvark)
- …
- And assigns each pair a probability:
- $P(+|w, c)$
- $P(-|w, c) = 1 - P(+|w, c)$

Skip-Gram Classifier

- **Positive class (+)** → c *is* a true context word
- **Negative class (–)** → c is *not* a true context word
- P(+ | apricot, jam) ≈ HIGH (close to 1)
- P(+ | apricot, aardvark) ≈ LOW (close to 0)
- Now the classifier learns:
- **For positive pairs:**
  - P(+ | apricot, jam) → 1
  - P(+ | apricot, of) → 1
  - P(+ | apricot, a) → 1
- **For negative pairs:**
  - P(+ | apricot, aardvark) → 0
  - P(+ | apricot, democracy)→ 0
- **Then automatically:**
  - P(– | apricot, jam) = 1 – 1 = 0
  - P(– | apricot, aardvark) = 1 – 0 = 1
- **For POSITIVE samples (real neighbors):**
  - Label = 1
  - P(+ | w, c) should be HIGH
  - P(– | w, c) = 1 – P(+ | w, c) should be LOW
- **For NEGATIVE samples (fake neighbors):**
  - Label = 0
  - P(+ | w, c) should be LOW
  - P(– | w, c) = 1 – P(+ | w, c) should be HIGH
- It updates the embeddings:
  - For positive pairs → move vectors closer
  - For negative pairs → push vectors apart
- So:
  - apricot **gets closer** to jam
  - apricot **gets closer** to “of” and “a”
  - apricot gets **pushed away** from aardvark
  - apricot gets **pushed away** from random negatives
- This is how embeddings capture semantic similarity.

Similarity is computed from dot product

- Remember: two vectors are similar if they have a high dot product
  - Cosine is just a normalized dot product
- So:
  - Similarity(w,c) $\propto w \cdot c$
- We'll need to normalize to get a probability
  - (cosine isn't a probability either)
- **Similarity(w, c) $\propto w \cdot c$**
  - The **similarity** between word **w** and context **c**
  - **Increases** when the **dot product** ($w \cdot c$) increases
  - **Decreases** when the dot product decreases
- But it does **NOT** mean:
  - Similarity(w, c) = $w \cdot c$
- Instead it means:
  - Similarity goes UP when dot product goes UP.
  - Similarity goes DOWN when dot product goes DOWN.

Turning dot products into probabilities

- The dot product **w·c** gives a number that can be:
  - very big (e.g., 6.8)
  - or very small (e.g., –9.3)
  - or anything between **–∞ to +∞**
- A probability **must be between 0 and 1**.
- So Skip-Gram with Negative Sampling (SGNS) uses the **sigmoid (logistic)** function:
- **1. Positive probability**
- $P(+|\, w, c) = \sigma(c \cdot w) = \frac{1}{1+\exp(-c \cdot w)}$
- If apricot and jam occur together → dot product big → P(+)=high.
- **2. Negative probability**
- $P(-|\, w, c) = 1 - P(+|\, w, c) = \sigma(-(c \cdot w)) = \frac{1}{1+\exp(c \cdot w)}$
- If apricot and aardvark do NOT occur together → dot product small → P(–)=high.
- **Dot product** tells how similar w and c are.
  - **Sigmoid** converts similarity into a probability.
  - **P(+|w,c)** uses **σ(c·w)** → high if similar.
  - **P(–|w,c)** uses **σ(–c·w)** → high if dissimilar.
- They always sum to **1** because it's a binary classification.

How Skip-Gram Classifier computes P(+|w, c)

- For ONE context word:

$$P(+|\, w, c) = \sigma(c \cdot w)$$

- For MANY context words $c_1, c_2, \ldots, c_L$:

$$P(+|\, w, c_{1:L}) = \prod_{i=1}^{L} \sigma(\, c_i \cdot w)$$

- And in log form:

$$\log P(+|\, w, c_{1:L}) = \sum_{i=1}^{L} \log \sigma(\, c_i \cdot w)$$

How Skip-Gram Classifier computes P(+|w, c)

- **1. Skip-gram tries to predict ALL context words around the target**
- Example:
- ... a tablespoon of APRICOT jam , a pinch ...
- If window = ±2, then **context words = L words**:
  - c1 = tablespoon
  - c2 = of
  - (c = apricot, target)
  - c3 = jam
  - c4 = a
- Skip-gram must learn:
  - apricot → tablespoon
  - apricot → of
  - apricot → jam
  - apricot → a
- **The model must successfully classify ALL of these as real neighbors.**

How Skip-Gram Classifier computes P(+|w, c)

- **2. Skip-gram assumes the context words are independent**
- This is a **simplifying assumption**.
- Skip-gram says:
  - *"The probability of seeing all context words is the product of their individual probabilities."*
- So:
- $P(+|\, w, c_{1:L}) = \prod_{i=1}^{L} \sigma(\, c_i \cdot w)$
- If events are independent:
  - $P(A \text{ and } B) = P(A) \cdot P(B)$
  - Event A = "tablepoon is real context of apricot"
  - Event B = "jam is real context of apricot"
- Skip-gram pretends these two events are independent (even though they aren't naturally).
- This is a **modeling trick** that makes training easy.

How Skip-Gram Classifier computes P(+|w, c)

- **3. Why take the log? (Very important)**
- Multiplying many probabilities can cause:
  - numbers to get too small
  - slow training
  - numerical underflow
- So we take the log:
- $\log P(+|\, w, c_{1:L}) = \sum_{i=1}^{L} \log\sigma(\, c_i \cdot w)$
- Logs turn multiplication → addition
  - Easy for gradient descent
  - Stable for computation
  - Faster training
- This is why neural networks typically optimize **log-probabilities**.
- **Imagine for "apricot" we want:**
  - P(apricot→jam) = 0.95
  - P(apricot→of) = 0.90
  - P(apricot→a) = 0.80
  - P(apricot→tablespoon) = 0.85
- Skip-gram multiplies them:
  - $0.95 \times 0.90 \times 0.80 \times 0.85$
- to get the full window probability.
- Taking logs:
  - $\log(0.95) + \log(0.90) + \log(0.80) + \log(0.85)$
- This is MUCH easier to optimize.

How Skip-Gram Classifier computes P(+|w, c)

- **Why does Skip-gram treat context words independently?**
- Because it makes training computationally cheap and fast.
- Skip-gram does **not** model grammar, order, or interactions between context words.
- It only cares that:
  - ALL context words are likely neighbors
  - but it doesn’t care how they interact
- This is why Word2Vec is so fast.

NUMERIC EXAMPLE

- We will use a **tiny embedding size = 2**
- **Step 0: Initial random embeddings**
- Let:
  - apricot → w = [0.20, 0.10]
  - tablespoon → c1 = [0.50, 0.20]
  - of → c2 = [0.10, 0.30]
  - jam → c3 = [0.40, 0.80]
  - a → c4 = [0.05, 0.10]
- We assume window = ±2, so context words are:
  - c1 = tablespoon
  - c2 = of
  - c3 = jam
  - c4 = a
- **Step 1: Compute dot products (similarity)**
- **1. (apricot · tablespoon)**
  - $0.20 * 0.50 + 0.10 * 0.20 = 0.10 + 0.02 = 0.12$
- **2. (apricot · of)**
  - $0.20 * 0.10 + 0.10 * 0.30 = 0.02 + 0.03 = 0.05$
- **3. (apricot · jam)**
  - $0.20 * 0.40 + 0.10 * 0.80 = 0.08 + 0.08 = 0.16$
- **4. (apricot · a)**
  - $0.20 * 0.05 + 0.10 * 0.10 = 0.01 + 0.01 = 0.02$
- So,
  - (apricot, jam): 0.16 ✅ highest (good neighbour)
  - (apricot, tablespoon): 0.12
  - (apricot, of): 0.05
  - (apricot, a): 0.02
  - Makes sense because “jam” is strongly related to “apricot.”

NUMERIC EXAMPLE

- **Step 2: Convert dot products into probabilities**
- Using sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$
- **1. P(+ | apricot, jam)**
  - $\sigma(0.16) = 0.5399 \approx 0.54$
  - **Model is not confident enough yet.**
- **2. P(+ | apricot, tablespoon)**
  - $\sigma(0.12) = 0.5299 \approx 0.53$
- **3. P(+ | apricot, of)**
  - $\sigma(0.05) = 0.5125 \approx 0.51$
- **4. P(+ | apricot, a)**
  - $\sigma(0.02) = 0.5050 \approx 0.51$

- **Which predictions are good? Which are wrong?**
  - We know from the real sentence that ALL are positive neighbors.
- But the model says:
  - apricot → jam → 1 → 0.54 ✅ good
  - apricot → tablespoon → 1 0.53 ✅ good
  - apricot → of → 1 → 0.51 ❌ too weak
  - apricot → a → 1 → ❌ too weak
- So two predictions are “strong,” two are “weak.”

NUMERIC EXAMPLE

- **Step 4: Update embeddings using gradient descent**
- For simplicity we use a small learning rate η = 0.1
- Update rule (positive samples):
- Skip-gram **updates BOTH vectors**:
  - $w := w + \eta(1 - \sigma(w \cdot c)) \cdot c$
  - $c := c + \eta(1 - \sigma(w \cdot c)) \cdot w$
  - The target word (apricot) moves closer to its context
  - The context word (jam) also moves closer to the target
- We will compute updates for each pair separately.
- **UPDATE 1: (apricot → jam)**
  - $error = 1 - 0.54 = 0.46$
- Update apricot:
  - w = w + 0.1 * 0.46 * c3
  - = [0.20, 0.10] + 0.046 * [0.40, 0.80]
  - = [0.20 + 0.0184, 0.10 + 0.0368]
  - = [0.2184, 0.1368]
- apricot becomes more like jam
- jam becomes more like apricot
- **UPDATE 2 — (apricot → tablespoon)**
  - $error = 1 - 0.53 = 0.47$
- Update apricot:
  - w = [0.2184, 0.1368] + 0.1 * 0.47 * [0.50, 0.20]
  - = [0.2184 + 0.0235, 0.1368 + 0.0094]
  - = [0.2419, 0.1462]
- apricot moves slightly toward "tablespoon"

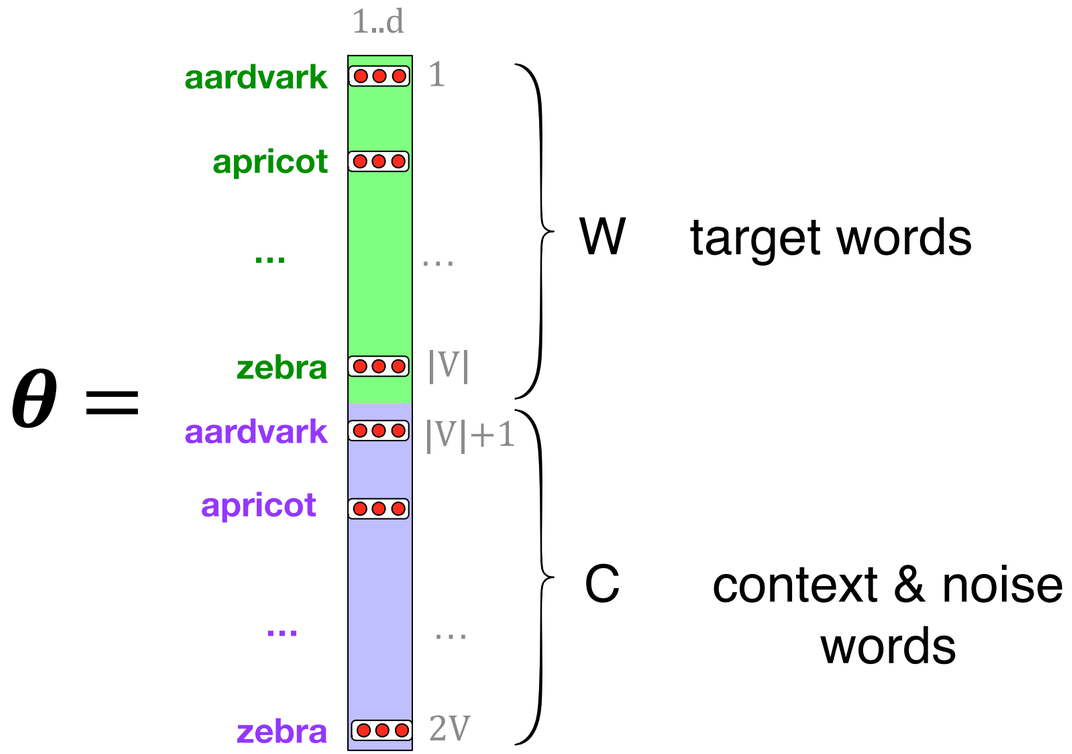
- **UPDATE 3 — (apricot → of)**
  - $error = 1 - 0.51 = 0.49$
- Update apricot:
  - w = [0.2419, 0.1462] + 0.1 * 0.49 * [0.10, 0.30]
  - = [0.2419 + 0.0049, 0.1462 + 0.0147]
  - = [0.2468, 0.1609]
- apricot moves toward "of"
- **UPDATE 4 — (apricot → a)**
  - $error = 1 - 0.51 = 0.49$
- Update apricot:
  - w = [0.2468, 0.1609] + 0.1 * 0.49 * [0.05, 0.10]
  - = [0.2468 + 0.00245, 0.1609 + 0.0049]
  - = [0.2493, 0.1658]
- apricot moves slightly toward "a"

NUMERIC EXAMPLE

- **FINAL UPDATED “apricot” embedding**
- **Before training:**
  - apricot = [0.20, 0.10]
- **After 4 independent context updates:**
  - apricot = [0.2493, 0.1658]
- It was pulled toward ALL four context words
  - Each prediction contributed
  - No pair was ignored
  - No “overall correctness” needed
  - Skip-gram just adds all updates together

These embeddings we'll need: a set for w, a set for c

- Here's the intuition of parameters we'll need.
- Skip-gram actually stores two embeddings for each word, one for the word as a target, and one for the word considered as context.
- Thus the parameters we need to learn are two matrices *W* and *C*, each containing an embedding for every one of the |*V* | words in the vocabulary.



These embeddings we'll need: a set for w, a set for c

- SGNS learns two sets of embeddings
  - Target embeddings matrix W
  - Context embedding matrix C
- It's common to just add them together, representing word *i* as the vector $w_i + c_i$
- **But if we want *one* final vector per word?**
- In practice, when you use Word2Vec for downstream tasks (classification, clustering, similarity, etc.), you usually **don’t want two vectors**, just one.
- **Common solution: Add them**
- You represent the word using:
- $v_i = w_i + c_i$
- Why add them?
  - Both w_i and c_i capture useful, complementary information.
  - Adding them merges the “target meaning” and “context meaning” into one richer vector.
  - It’s simple and works well in practice.

SGNS learns two sets of embeddings

- SGNS learns two sets of embeddings
  - Target embeddings matrix W
  - Context embedding matrix C
- It's common to just add them together, representing word *i* as the vector $w_i + c_i$
- **But if we want *one* final vector per word?**
- In practice, when you use Word2Vec for downstream tasks (classification, clustering, similarity, etc.), you usually **don't want two vectors**, just one.
- **Common solution: Add them**
- You represent the word using:
- $v_i = w_i + c_i$
- Why add them?
  - Both w_i and c_i capture useful, complementary information.
  - Adding them merges the "target meaning" and "context meaning" into one richer vector.
  - It's simple and works well in practice.

Summary: How Word2Vec (Skip-Gram) Learns Embeddings

- **Start with random vectors**
  - You create **V** word vectors (one for each word in the vocabulary).
  - Each vector has **d dimensions** (like 100, 200, or 300).
  - These are random at first.
- **Build a classifier based on similarity**
  - The classifier's job is to decide:
    "Do these two words occur together in real text?"
- **Create training examples from a corpus**
  - **Positive examples:** pairs of words that appear close to each other
    (e.g., "cat–sat", "king–royal").
  - **Negative examples:** random pairs that don't usually appear together
    (e.g., "cat–banana", "river–phone").
- **Train the classifier**
  - If the pair is a real context pair → classifier should output **1**
  - If it's a fake pair → classifier should output **0**
  - During this process, the model **adjusts all word vectors** so that:
    a. Real pairs have **higher similarity**
    b. Fake pairs have **lower similarity**
- **Throw away the classifier**
  - Once training is done, you **don't need the classifier anymore**.
  - The useful output is the **trained word embeddings**.

Vector Semantics & Embeddings

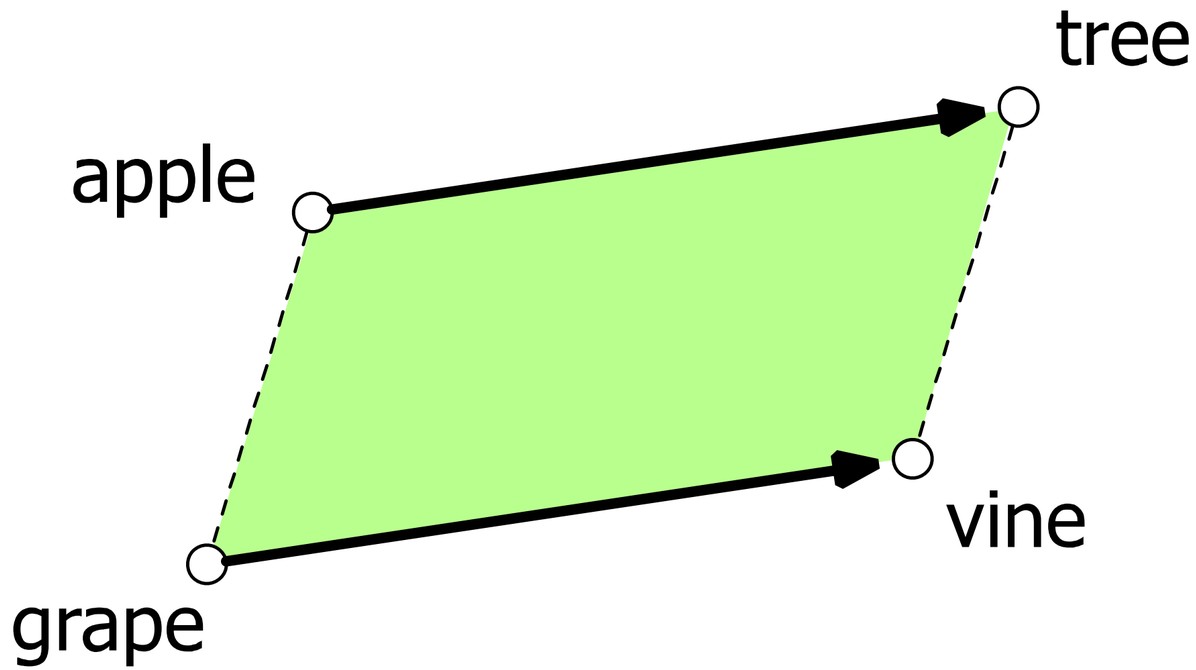
Properties of Embeddings

How Context Window Size Shapes Word Similarity

- The **window size** in Word2Vec (skip-gram or CBOW) controls *how many words around the target word* are used as context.
- Typical window size:
  - **1–10 words on each side** (so total context = 2–20 words)
- The choice of window strongly affects what kind of neighbors you get in the embedding space.
- **1. Small window (±2) → Syntactic and similar-type words**
- A small window looks only at immediate neighbors. These tend to be:
  - Words with **similar syntactic roles**
  - Words of the **same type/category**
  - Often **semantically similar**
- **2. Large window (±5) → Semantic or topical neighbors**
- A larger window includes words from the wider topic or theme, not just grammar-related neighbors.
- These neighbors tend to be:
  - **Related by topic**
  - **Part of the same world or domain**
  - **Associated concepts**, not similar types
- **Practical takeaway**
- **Small window = "similar words"** (same type/role)
- **Large window = "related words"** (same topic/domain)

How Context Window Size Shapes Word Similarity

- The classic parallelogram model of analogical reasoning (Rumelhart and Abrahamson 1973)
- To solve: *"apple is to tree as grape is to _____"*
- *Add tree – apple to grape to get vine*



How Context Window Size Shapes Word Similarity

- Word embeddings don't just know what words *mean,* they also capture **relationships between words**.
- A famous idea that explains this is the **parallelogram model** (Rumelhart & Abrahamson, 1973).
- **apple : tree :: grape : ?**
  ("apple is to tree as grape is to what?" → answer: **vine**)
- We want the model to understand the relationship:
  - **apple grows on a tree**
  - **grape grows on a vine**
- **How embeddings solve this analogy**
- Each word is a point in a vector space.
- **Step 1: Find the relationship between the first pair**
- Compute the vector difference: tree – apple
- This vector represents the relationship: "what an apple grows on"
- **Step 2: Apply this relationship to the second word**
- Add that relationship to **grape**:
- grape + (tree – apple)
- This gives a new point in the space.
- **Step 3: Find the closest word**
- The nearest word to that point is usually: **vine**
- **Why this works (intuitively)**
- Think of vectors like arrows.
  The arrow from **apple** → **tree** is similar to the arrow from **grape** → **vine**.
- Visually, these two arrows form a **parallelogram shape** that's why it's called the *parallelogram model*.
- So embeddings capture:
  - not just meanings of words
- but **relationships between them**, like:
  - capital of…
  - male → female
  - singular → plural
  - fruit → plant it grows on
- Word embeddings let us treat words as points in space.
- The difference between two word vectors captures a **relationship**.
- To solve an analogy, we apply that relationship to a third word.
- The nearest word to the result is the answer.
- That's how embeddings do analogies like:
  - king – man + woman ≈ queen
  - Paris – France + Italy ≈ Rome
  - apple – tree + grape ≈ vine

Analogical relations via parallelogram

- The parallelogram method can solve analogies with both sparse and dense embeddings (Turney and Littman 2005, Mikolov et al. 2013b)
- king – man + woman is close to queen
- Paris – France + Italy is close to Rome
- An analogy has the form: **a : a* :: b : b***
- Examples:
- **king : man :: queen : woman**
- **Paris : France :: Rome : Italy**
- We want to find **b***.

Analogical relations via parallelogram

- **Step-by-step idea**
- **Find the relationship between a and a***
  - Subtract their vectors: $a^* - a$
- This vector represents the meaning of the relationship ("man → woman", "France → Italy", etc.)
- **Apply the same relationship to b**
  - Add this relationship to b: $a^* - a + b$
- **Find the word closest to that point** in vector space

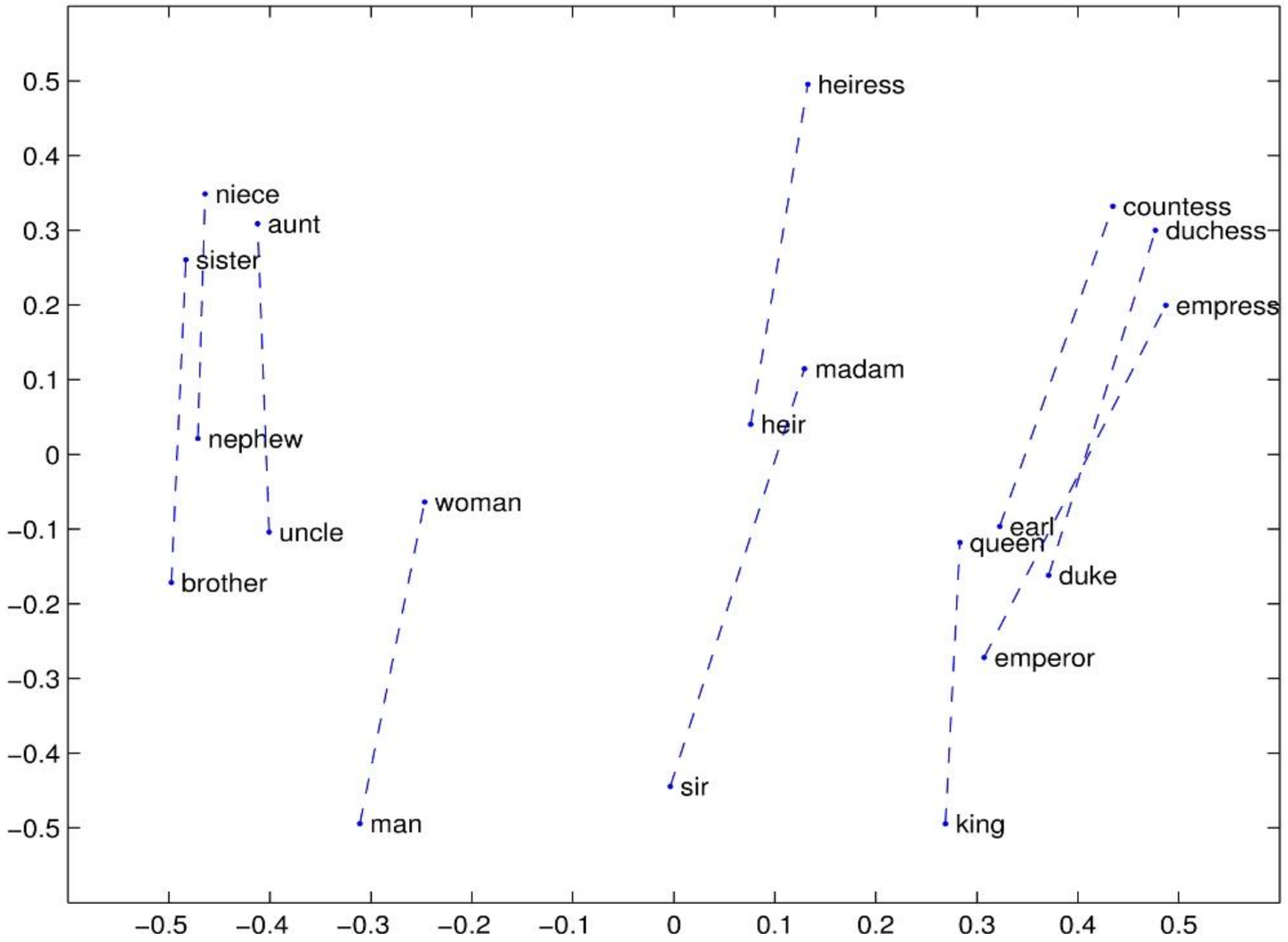
  That word is your predicted answer b^*.
- **Formula (simplified):**
- $\widehat{b^*} = \arg\max_x \text{ distance}(x,\ a^* - a + b)$
- This means:
- "Find the word **x** whose vector is closest to the point **a* − a + b**."
- **Simple examples**
- **Example 1:**
- **king − man + woman ≈ queen**
- king → man = "male form"
- Apply similar shift to woman → "female form"
- Nearest word = **queen**
- **Example 2:**
- **Paris − France + Italy ≈ Rome**
- Relationship:

  France → Paris

  Italy → ?
- Apply the same relation:

  Nearest vector ≈ **Rome**
- **Why it's called the parallelogram method**
- If you draw the vectors, the relationships form the sides of a **parallelogram**:
- The shift from *king → queen* is parallel to *man → woman*
- The shift from *France → Paris* is parallel to *Italy → Rome*
- Word embeddings naturally align these relationships.

Analogical relations via parallelogram



Word2 Vec Research Paper

- **1. Primary Paper (Efficient Estimation):**
  - Title: Efficient Estimation of Word Representations in Vector Space
  - Authors: Tomas Mikolov, Kai Chen, Greg Corrado, Jeffrey Dean
  - Link: https://arxiv.org/abs/1301.3781
  - Key Contribution: Introduces the Continuous Bag-of-Words (CBOW) and Skip-gram architectures.
- **2. Follow-up Paper (Extensions/Optimizations):**
  - Title: Distributed Representations of Words and Phrases and their Compositionality
  - Authors: Tomas Mikolov, Ilya Sutskever, Kai Chen, Greg Corrado, Jeffrey Dean
  - Link: arXiv: https://arxiv.org/abs/1310.4546
  - Key Contribution: Introduces Negative Sampling (NEG) and subsampling of frequent words for faster training and better vector quality.
- Original Code Implementation:
  - https://code.google.com/archive/p/word2vec/

Caveats with the parallelogram method

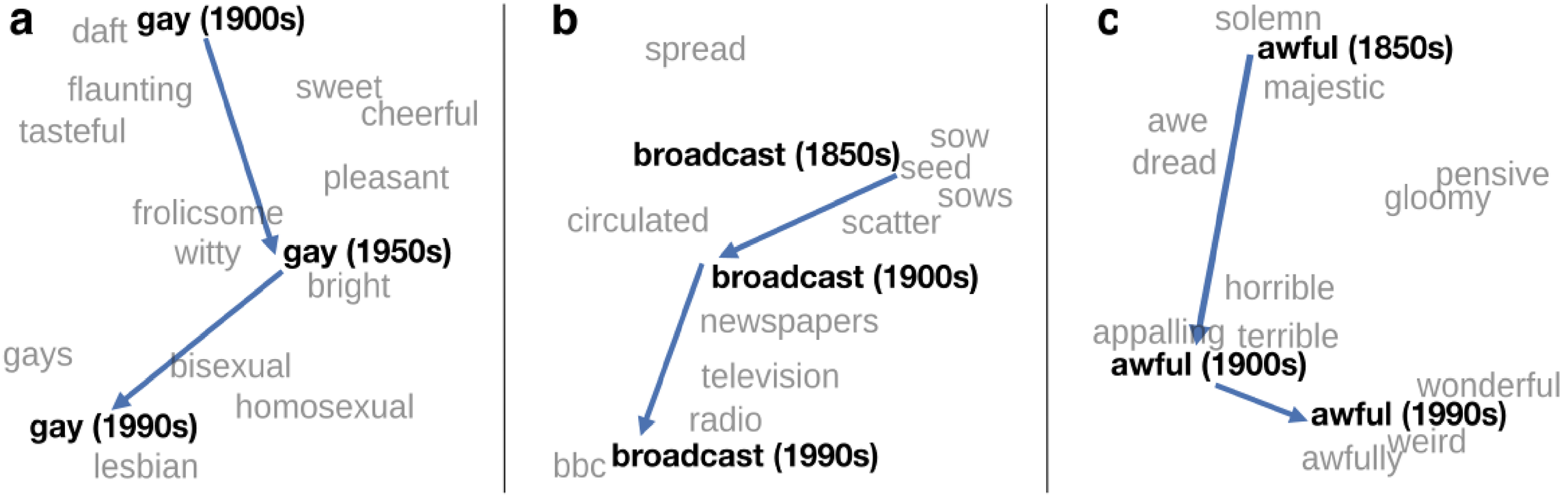
- It only seems to work for frequent words, small distances and certain relations (relating countries to capitals, or parts of speech), but not others. (Linzen 2016, Gladkova et al. 2016, Ethayarajh et al. 2019a)

- Understanding analogy is an open area of research (Peterson et al. 2020)

Embeddings as a window onto historical semantics

Train embeddings on different decades of historical text to see meanings shift

~30 million books, 1850-1990, Google Books data



William L. Hamilton, Jure Leskovec, and Dan Jurafsky. 2016. Diachronic Word Embeddings Reveal Statistical Laws of Semantic Change. Proceedings of ACL.

Embeddings reflect cultural bias!

- Ask “Paris : France :: Tokyo : x”
  - x = Japan

Bolukbasi, Tolga, Kai-Wei Chang, James Y. Zou, Venkatesh Saligrama, and Adam T. Kalai. "Man is to computer programmer as woman is to homemaker? debiasing word embeddings." In *NeurIPS*, pp. 4349-4357. 2016.

- Ask “father : doctor :: mother : x”
  - x = nurse
- Ask “man : computer programmer :: woman : x”
  - x = homemaker

Algorithms that use embeddings as part of e.g., hiring searches for programmers, might lead to bias in hiring

Historical embedding as a tool to study cultural biases

Garg, N., Schiebinger, L., Jurafsky, D., and Zou, J. (2018). Word embeddings quantify 100 years of gender and ethnic stereotypes. Proceedings of the National Academy of Sciences 115(16), E3635–E3644.

- Compute a **gender or ethnic bias** for each adjective: e.g., how much closer the adjective is to "woman" synonyms than "man" synonyms, or names of particular ethnicities
  - Embeddings for **competence** adjective (*smart, wise, brilliant, resourceful, thoughtful, logical)* are biased toward men, a bias slowly decreasing 1960-1990
  - Embeddings for **dehumanizing** adjectives (barbaric, monstrous, bizarre) were biased toward Asians in the 1930s, bias decreasing over the 20th century.
- These match the results of old surveys done in the 1930s